



Escola Superior de Tecnologia e Gestão
Licenciatura em Engenharia Informática

CAPACITA – Medição de distâncias entre segmentos do corpo humano com métodos de visão por computador

Artem Bukhantsev

Beja, Setembro de 2024

INSTITUTO POLITÉCNICO DE BEJA

Escola Superior de Tecnologia e Gestão

Licenciatura em Engenharia Informática

**CAPACITA – Medição de distâncias entre segmentos
do corpo humano com métodos de visão por
computador**

Artem Bukhantsev

Orientado por:

Professor Luís Carlos da Silva Bruno, IPBeja

Relatório de projeto de fim de curso apresentado na
Escola Superior de Tecnologia e Gestão do Instituto Politécnico de Beja

Resumo

CAPACITA – Medição de distâncias entre segmentos do corpo humano com métodos de visão por computador

O objetivo deste Projeto de Curso Final é abordar o significativo envelhecimento das populações nas sociedades ocidentais, particularmente na região do Alentejo, desenvolvendo um sistema que mitiga o declínio das capacidades físicas através de iniciativas de envelhecimento ativo. Este projeto foca na automação do processo de recolha de dados para exercícios físicos baseados na Bateria de Fullerton (BF), uma ferramenta de avaliação amplamente reconhecida nesta área. Utilizando Python em conjunto com as bibliotecas Mediapipe e PyKinect, o sistema detecta com precisão partes do corpo humano e mede as distâncias entre elas utilizando imagens capturadas pela câmara RGB do Kinect. Nos testes realizados, a implementação Mediapipe demonstrou desempenho superior, alcançando um erro médio absoluto (MAE) de aproximadamente 2,25 cm, comparado aos 8,65 cm da implementação PyKinect. A abordagem Mediapipe foi eficaz no teste “Sentado e alcançar os pés com as mãos”, garantindo avaliações automatizadas fiáveis e precisas. No entanto, o teste “Alcançar as mãos atrás das costas” não pôde ser implementado devido a limitações tecnológicas.

Palavras-chave: Python, Mediapipe, PyKinect, Envelhecimento ativo, Visão por computador, Bateria de Fullerton, Segmentação do corpo humano, Medição de distâncias, Cuidados com idosos, Avaliação automatizada..

Abstract

Measurement of distances between segments of the human body using computer vision methods

The aim of this Final Course Project is to address the significant aging of populations in Western societies, particularly in the Alentejo region, by developing a system that mitigates the decline in physical capabilities through active aging initiatives. This project focuses on automating the data collection process for physical exercises based on the Fullerton Battery (FB), a widely recognized assessment tool in this field. The system is designed for deployment in various settings, including homes, day centers, gyms, and elderly residences. Using Python in conjunction with the Mediapipe and PyKinect libraries, the system accurately detects human body parts and measures the distances between them using images captured by the Kinect. In the conducted tests, the Mediapipe implementation demonstrated superior performance, achieving a mean absolute error (MAE) of approximately 2.25 cm compared to 8.65 cm for the PyKinect implementation. Specifically, the "Sitting and reaching the feet with the hands" test was successfully automated and evaluated using Mediapipe. However, the "Reaching the hands behind the back" test could not be implemented due to technological limitations. This approach ensures reliable and precise automated assessments, thereby enhancing the effectiveness of active aging programs.

Keywords: Python, Mediapipe, PyKinect, Active aging, Computer vision, Fullerton Battery, Human body segmentation, Distance measurement, Elderly care, Automated assessment.

Acknowledgments

I would like to express my sincere gratitude to everyone who contributed to the completion of this Final Course Project.

First and foremost, I extend my heartfelt thanks to Professor Luís for his invaluable guidance, continuous support, and the enriching discussions that were fundamental to the development of this project. His expertise and dedication were essential to the realization of this work.

Additionally, I would like to thank all the participants who contributed to the system tests. Your willingness to dedicate time and provide detailed feedback was crucial in enhancing the accuracy and effectiveness of the developed solution.

Contents

Resumo	i
Abstract	ii
Acknowledgments	iii
Contents	iv
List of Figures	viii
List of Listings	x
List of Tables	xii
1 Introduction	1
1.1 Project Objectives	2
1.2 Significance of the Study	2
1.3 Key Findings	2
1.4 Structure of the Document	3
2 Background	5
2.1 The CAPACITA Project	5
2.1.1 Overview	5
2.1.2 Motivation	5
2.1.3 Objectives	6
2.1.4 Relevance to the Current Project	6
2.2 The Fullerton Battery	6
2.2.1 Introduction	7
2.2.2 Components of the Fullerton Battery	7
2.2.3 Importance of Accurate Measurement	10

2.3	Technologies Used	10
2.3.1	Computer Vision in Health Assessments	11
2.3.2	Mediapipe	11
2.3.3	PyKinect and the Kinect device	12
2.4	System Requirements	15
2.4.1	Functional Requirements	15
2.4.2	Technical Requirements	15
2.5	Specific Objectives	16
2.5.1	Objective 1: Develop Measurement Algorithms	16
2.5.2	Objective 2: Implement Two Program Versions	17
2.5.3	Objective 3: Validate and Test the Programs	18
3	System Architecture and Setup Details	19
3.1	System Architecture	19
3.1.1	Hardware Components	19
3.1.2	Software Components	20
3.1.3	System Workflow	21
3.1.4	Data Flow and Processing	22
3.2	Setup Details	23
3.2.1	Software Environment Setup	23
3.2.2	Hardware Setup	27
3.2.3	Running the Code	27
3.2.4	Troubleshooting Common Issues	28
3.2.5	Environmental Setup	29
3.2.6	Device Positioning	30
3.2.7	User Positioning	31
3.3	Code Implementations	32
3.3.1	Mediapipe Version with Kinect Input	33
3.3.2	PyKinect Version	43
3.3.3	Comparison of Mediapipe and PyKinect Implementations	51
3.3.4	Challenges and Considerations	52
4	Testing and Validation	53
4.1	Testing Environment and Setup	53
4.1.1	Kinect device Setup	53
4.1.2	Computer System Specifications	56

4.1.3	Software Environment	57
4.1.4	Participant Preparation	57
4.1.5	Testing Protocol	57
4.2	Participant Demographics	58
4.3	Experimental Procedure	58
4.3.1	Chair Sit and Reach Test	59
4.3.2	Data Processing	59
4.4	Results and Analysis	60
4.4.1	Accuracy Metrics	60
4.4.2	Mediapipe Implementation Results	61
4.4.3	PyKinect Implementation Results	63
4.4.4	Comparative Analysis	66
4.4.5	Participant-Specific Observations	67
4.5	Challenges with Back Scratch Test	68
4.5.1	Limitations of the Implementations	68
4.5.2	Technical Attempts to Mitigate Issues	68
4.5.3	Conclusion on Back Scratch Test	69
4.6	Discussion	70
4.6.1	Advantages of Mediapipe Implementation	70
4.6.2	Limitations of PyKinect Implementation	71
4.6.3	Factors Affecting Accuracy	71
4.6.4	Recommendations for Improvement	71
4.7	Conclusion	72
4.7.1	Key Findings	72
4.7.2	Implications	72
4.7.3	Future Work	72
5	Conclusions and Future Work	74
5.1	Summary of Research	74
5.2	Key Findings	75
5.2.1	Mediapipe Implementation Superiority	75
5.2.2	Limitations of PyKinect Implementation	75
5.2.3	Challenges with Back Scratch Test	75
5.3	Implications for Human Movement Analysis	76
5.3.1	Feasibility of 2D Pose Estimation	76

5.3.2	Limitations of Depth-Based Systems	76
5.3.3	Importance of Algorithm Training Data	76
5.3.4	Potential for Accessible Assessment Tools	76
5.4	Limitations of the Study	77
5.4.1	Sample Size and Diversity	77
5.4.2	Single Functional Test Focus	77
5.4.3	Environmental Control	77
5.4.4	Calibration Procedures	77
5.5	Recommendations for Future Work	77
5.5.1	Enhancing Pose Estimation Models	77
5.5.2	Improving Measurement Accuracy	78
5.5.3	User Interface and Experience Enhancements	78
5.5.4	Integration with Machine Learning and AI	78
5.5.5	Clinical and Rehabilitation Applications	79
5.5.6	Multi-Sensor and Multi-Modal Approaches	79
5.6	Final Remarks	79
Bibliography		80
A Appendix		82
A.1	Mediapipe Implementation	82
A.2	PyKinect Implementation	91

List of Figures

2.1	Original Capacita system description	6
2.2	Chair Stand test visual example	7
2.3	Arm Curl Test visual example	8
2.4	Chair Sit-and-Reach Test visual example	8
2.5	Back Scratch Test visual example	9
2.6	8-Foot Up-and-Go Test visual example	9
2.7	6-Minute Walk Test visual example	10
2.8	Mediapipe skeleton estimation example showing body landmarks [10]	12
2.9	Microsoft Kinect skeleton estimation example showing body landmarks [13]	13
2.10	Microsoft Kinect device used for depth sensing and skeletal tracking [13]	14
2.11	Chair Sit-and-Reach visual representation	16
2.12	Back Scratch visual representation	17
3.1	System architecture workflow diagram illustrating the sequence of processes from data acquisition to feedback visualization	22
3.2	Optimal Kinect device positioning and alignment for accurate motion capture	30
3.3	User positioning relative to the device, ensuring full-body capture and optimal distance	32
3.4	Flowchart of the Mediapipe implementation using Kinect camera	42
3.5	Flowchart of the PyKinect implementation	50
4.1	Kinect device Setup for Testing	55
4.2	Kinect device Setup for Testing Height	56
4.3	Bland-Altman Plot for Mediapipe Implementation	62
4.4	Bland-Altman Plot for PyKinect Implementation	64
4.5	Skeleton Jittering Observed in PyKinect Implementation	65
4.6	Skeleton Jittering Observed in PyKinect Implementation Another Example	66

4.7	Comparison of Absolute Errors between Mediapipe and PyKinect Implementations	66
4.8	Landmark Detection Failures in Back Scratch Test	69
4.9	Landmark Detection Failures in Back Scratch Test N2	70

List of Listings

3.1	Test script to verify imports	26
3.2	Test script to verify Kinect device	27
3.3	Importing required libraries	33
3.4	Initializing Kinect and Mediapipe Pose	33
3.5	Function to calculate Euclidean distance in 2D	34
3.6	Function to adjust fingertip position	34
3.7	Function to calculate angle between three points	35
3.8	Function to calculate flexibility score	35
3.9	Function to classify flexibility score	37
3.10	Function to draw dynamic angle arc	37
3.11	Function to draw progress bar	38
3.12	Part of function to draw flexibility bar	38
3.13	Function to calculate average distance	39
3.14	Main loop for frame processing	39
3.15	Part of pose estimation and calculations	40
3.16	Part of visualization and feedback	40
3.17	Displaying final results	41
3.18	Importing required libraries	43
3.19	Initializing Kinect device	44
3.20	Function to calculate Euclidean distance in 3D	44
3.21	Function to calculate angle between three points in 3D	44
3.22	Function to draw progress bar	45
3.23	Part of function to draw flexibility bar	45
3.24	Function to classify flexibility score	46
3.25	Function to calculate flexibility score	46

3.26	Part of function to draw skeleton	47
3.27	Part of main loop for Kinect data processing	48
3.28	Part of pose estimation and calculations	49
3.29	Visualization and feedback	49
3.30	Displaying final results	50
A.1	Full script of Mediapipe	82
A.2	Full script of PyKinect	91

List of Tables

4.1 Participant Demographics	58
4.2 Mediapipe Implementation Results	61
4.3 PyKinect Implementation Results	63

Chapter 1

Introduction

The phenomenon of population aging is a global trend with profound implications for societies worldwide [17]. In regions like the Alentejo in Portugal, the demographic shift towards an older population is particularly pronounced due to factors such as low birth rates and youth migration to urban areas. This shift results in a higher proportion of elderly individuals, which poses challenges for healthcare systems, social services, and community support structures.

As people age, they often experience a natural decline in physical capabilities, including strength, flexibility, balance, and endurance [17]. These declines can lead to decreased independence, reduced quality of life, and increased risk of injuries from falls or other accidents. Early detection and assessment of physical limitations are crucial for implementing interventions that promote active aging and help maintain functional independence.

To address these challenges, the CAPACITA project was initiated with the aim of developing an automated system for measuring distances between segments of the human body using computer vision methods [1]. CAPACITA facilitates accurate detection and measurement of human body parts during physical exercises, specifically those based on the Fullerton Functional Fitness Test (FFFT) [11]. By automating the data collection and analysis process, CAPACITA enables healthcare professionals and caregivers to monitor physical decline more effectively, tailor interventions to individual needs, and ultimately promote active aging among the elderly population in the Alentejo region and beyond.

1.1 Project Objectives

The primary objective of this project is to develop an accurate and reliable program capable of measuring distances between human body segments using computer vision methods. This program aims to automate the assessment of specific physical fitness tests from the Fullerton Battery (FB), namely the "Chair Sit-and-Reach" and "Back Scratch" tests.

To achieve this, the project involves:

- Developing two versions of the measurement program using different technologies:
 - **Version 1:** Utilizing Mediapipe [10] and Python to detect body landmarks via standard RGB cameras.
 - **Version 2:** Employing PyKinect [4] and the Kinect device to leverage depth data and skeletal tracking.
- Ensuring high accuracy and reliability in the measurements to match or exceed manual assessment methods.
- Conducting extensive testing and validation to confirm the program's effectiveness across various conditions and participants.

1.2 Significance of the Study

By automating the measurement process and improving accuracy, the project contributes to:

- **Enhanced Assessments.** Providing healthcare professionals with precise tools for evaluating functional fitness in the elderly.
- **Active Aging Promotion.** Supporting initiatives that encourage older adults to engage in physical activities tailored to their capabilities.
- **Technological Advancement.** Demonstrating the practical application of computer vision technologies in health assessments without the need for complex user interfaces.

1.3 Key Findings

During the testing phase, the Mediapipe implementation demonstrated significantly superior performance compared to the PyKinect implementation. Specifically, Mediapipe achieved an average absolute error

(MAE) of approximately 2.25 cm, markedly lower than the 8.65 cm recorded by PyKinect. This high level of accuracy was consistent across various participants, as indicated by a low standard deviation.

Additionally, Mediapipe exhibited exceptional sensitivity to small distances, which is critical for detecting subtle changes in flexibility. The system provided real-time responsiveness, delivering smooth feedback without noticeable lag, thereby enhancing the user experience.

Furthermore, Mediapipe proved to be robust against environmental variations, such as changes in lighting and background clutter, owing to its reliance on 2D pose estimation. In contrast, the PyKinect implementation faced several challenges, including higher error rates, skeletal jittering leading to inconsistent measurements, latency issues due to the computational demands of processing depth data, and greater sensitivity to environmental conditions.

These findings highlight the reliability and precision of the Mediapipe-based automated assessments, underscoring their potential to significantly improve the effectiveness of active aging programs. Specifically, the “Sitting and reaching the feet with the hands” test was successfully automated and evaluated using the Mediapipe implementation, ensuring reliable and precise measurements. However, the “Reaching the hands behind the back” test could not be implemented due to technological limitations related to skeletal tracking accuracy in that specific movement pattern.

1.4 Structure of the Document

The remainder of this document is structured as follows:

1. **Chapter 1: Introduction** – Introduces the project, outlining the motivation, objectives, scope, and the challenges addressed by the system.
2. **Chapter 2: Background** – Provides detailed explanations of the CAPACITA project, the Fullerton Functional Fitness Test (FFFT), the importance of active aging initiatives, and the technologies used, including Mediapipe and PyKinect.
3. **Chapter 3: System Architecture** – Describes the overall system design, including hardware and software components. It details the Kinect device setup (distance, angle, lighting conditions, background), PC specifications, and the environment configuration necessary for accurate data collection.
4. **Chapter 4: Analysis and Implementation** – Discusses the methodologies, algorithms, and technical considerations involved in developing both program versions. It includes detailed explanations of how distance measurements are performed using computer vision techniques.

-
5. **Chapter 5: Testing and Validation** – Describes the procedures for testing the programs, presents the results obtained, and analyzes the accuracy and reliability of the measurements compared to traditional methods. It includes participant demographics, experimental setup, and statistical analysis of the results.
 6. **Chapter 6: Conclusions and Future Work** – Summarizes the findings, discusses the implications of the results, and suggests potential areas for further research and development to enhance the system's capabilities.
 7. **Appendices** – Contains supplementary materials, including the full code listings of the implementations, additional resources, and any relevant documentation or configuration files.

Chapter 2

Background

In this chapter, we provide comprehensive explanations of all concepts and technologies relevant to this project. This includes detailed information on the CAPACITA initiative, an in-depth look at the Fullerton Battery, thorough descriptions of Mediapipe and PyKinect, the Kinect device's functionalities, system requirements, specific objectives, and the significance of this study.

2.1 The CAPACITA Project

In this chapter, provided explanations of all concepts and technologies relevant to this project. This includes information on the CAPACITA initiative, an in-depth look at the Fullerton Battery, thorough descriptions of Mediapipe and PyKinect, the Kinect device's functionalities, system requirements, specific objectives, and the significance of this study.

2.1.1 Overview

The CAPACITA project is an initiative developed at the Laboratory of Information Systems and Interactivity (LabSI2) at the Polytechnic Institute of Beja [1]. Since its inception in 2018, the project has focused on creating tools and methodologies to assess and enhance the physical and functional capacities of the elderly population.

2.1.2 Motivation

The motivation for the CAPACITA project arises from the need to address the challenges associated with an aging population. As individuals age, maintaining physical fitness becomes crucial for preserv-

ing independence and quality of life. Traditional assessment methods can be time-consuming, require specialized equipment, and may not be readily accessible in all settings.

2.1.3 Objectives

The objectives of the CAPACITA project include:

- Developing accessible and accurate assessment tools that can be used in various environments, such as homes, community centers, and clinics.
- Automating data collection processes to reduce reliance on manual measurements, thereby minimizing errors and inconsistencies.
- Providing data-driven insights to help design personalized exercise programs that cater to the specific needs of elderly individuals.

Figure 2.1 illustrates the CAPACITA system layout.

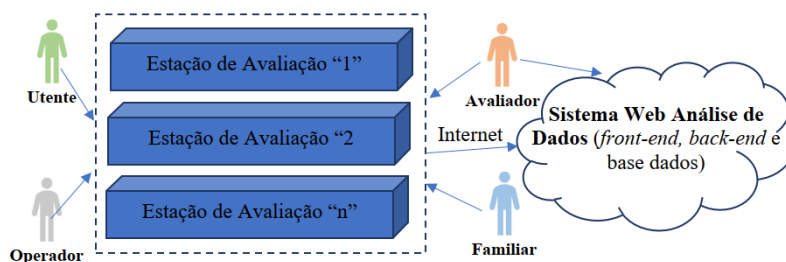


Figure 2.1: Original Capacita system description

2.1.4 Relevance to the Current Project

This project contributes to the CAPACITA initiative by focusing on the development of a program that accurately measures distances between body segments, which is essential for assessing certain tests in the Fullerton Battery [1]. By using the logic available within the CAPACITA framework, we aim to improve the assessment process and support better health outcomes for the elderly population.

2.2 The Fullerton Battery

The Fullerton Battery is a comprehensive set of assessments designed to evaluate the functional fitness of older adults. By systematically measuring various aspects of physical health, it provides critical insights into an individual's ability to perform daily activities with strength, flexibility, and balance. This section

delves into the specific tests that constitute the Fullerton Battery, their significance in assessing physical capabilities, and their relevance to the overall objectives of this study.

2.2.1 Introduction

The Fullerton Battery (FB), also known as the Senior Fitness Test, is a set of standardized physical tests designed to assess the functional fitness of older adults. Developed by researchers at California State University [11], Fullerton, the FB evaluates key components of physical fitness that are necessary for performing everyday activities safely and independently.

2.2.2 Components of the Fullerton Battery

The FB includes the following tests:

- **Chair Stand Test**

This test measures lower body strength by counting the number of times an individual can rise from a seated position to a full stand and return to a seated position within 30 seconds. Lower body strength is critical for tasks such as climbing stairs, getting out of a car, or rising from a chair. An example can be found in Figure 2.2



Figure 2.2: Chair Stand test visual example

- **Arm Curl Test**

This test assesses upper body strength by counting the number of bicep curls completed in 30 seconds while holding a hand weight. Upper body strength is essential for carrying groceries, lifting objects, or performing household tasks. An example can be found in Figure 2.3



Figure 2.3: Arm Curl Test visual example

- **Chair Sit-and-Reach Test**

This test evaluates lower body flexibility by measuring the distance between the fingertips and toes while the participant reaches forward from a seated position with one leg extended. Flexibility in the lower body affects mobility and the ability to perform activities like bending to tie shoes or pick up objects from the floor. An example can be found in [Figure 2.4](#)



Figure 2.4: Chair Sit-and-Reach Test visual example

- **Back Scratch Test**

This test assesses upper body (shoulder) flexibility by measuring the distance between the middle fingers when reaching one hand over the shoulder and the other up the middle of the back. Shoulder flexibility impacts the ability to dress, comb hair, or reach for objects overhead. An example can be found in [Figure 2.5](#)



Figure 2.5: Back Scratch Test visual example

- **8-Foot Up-and-Go Test**

This test measures agility and dynamic balance by timing how quickly an individual can rise from a seated position, walk eight feet, turn around, and return to the chair. Agility and balance are crucial for preventing falls and navigating obstacles. An example can be found in [Figure 2.6](#)



Figure 2.6: 8-Foot Up-and-Go Test visual example

- **6-Minute Walk Test**

This test evaluates aerobic endurance by measuring the distance an individual can walk in six minutes. Aerobic endurance is important for activities that require sustained effort, such as walking distances or performing continuous tasks. An example can be found in [Figure 2.7](#)



Figure 2.7: 6-Minute Walk Test visual example

2.2.3 Importance of Accurate Measurement

Accurate measurements are vital for:

- **Assessment Validity**

Ensuring that the assessments accurately reflect the individual's physical capabilities.

- **Monitoring Progress**

Tracking changes over time to evaluate the effectiveness of interventions or exercise programs.

- **Personalized Interventions**

Identifying specific areas of weakness that can be targeted with tailored exercises or therapies.

Traditional methods of measurement can introduce variability due to human error, differences in technique, or equipment limitations. Automating the measurement process using computer vision technologies can enhance accuracy, consistency, and efficiency.

2.3 Technologies Used

This section provides an in-depth examination of the technologies employed to achieve accurate measurement of distances between human body segments using computer vision methods. It covers the selection and utilization of key software frameworks, namely Mediapipe and PyKinect, which are instrumental in body landmark detection and pose estimation.

2.3.1 Computer Vision in Health Assessments

Computer vision is a field of artificial intelligence that enables computers to interpret and understand visual information from the world. In health assessments, computer vision techniques can be used to analyze movements, detect body landmarks, and measure physical parameters without the need for direct contact or invasive procedures.

2.3.2 Mediapipe

Mediapipe plays a pivotal role in our computer vision system by providing robust tools for real-time pose estimation and landmark detection. Its efficient framework enables accurate tracking of human body movements, which is essential for measuring distances between body segments during fitness assessments.

Overview

Mediapipe is an open-source framework developed by Google that provides solutions for building perception pipelines [14]. It allows for the implementation of cross-platform, customizable, and efficient computer vision applications.

Pose Estimation

Mediapipe's pose estimation solution, as illustrated in Figure 2.8, detects 33 key landmarks on the human body in real-time [14]. These landmarks correspond to anatomical points such as joints and extremities, which are essential for analyzing human posture and movement.

Advantages for This Project

Using Mediapipe offers several benefits:

- **No Specialized Hardware Required**

Mediapipe can operate using standard RGB cameras, making it accessible for widespread use.

- **Real-Time Processing**

The framework is optimized for real-time performance, which is essential for live assessments.

- **High Accuracy**

Mediapipe's advanced algorithms provide precise detection of body landmarks under various conditions.

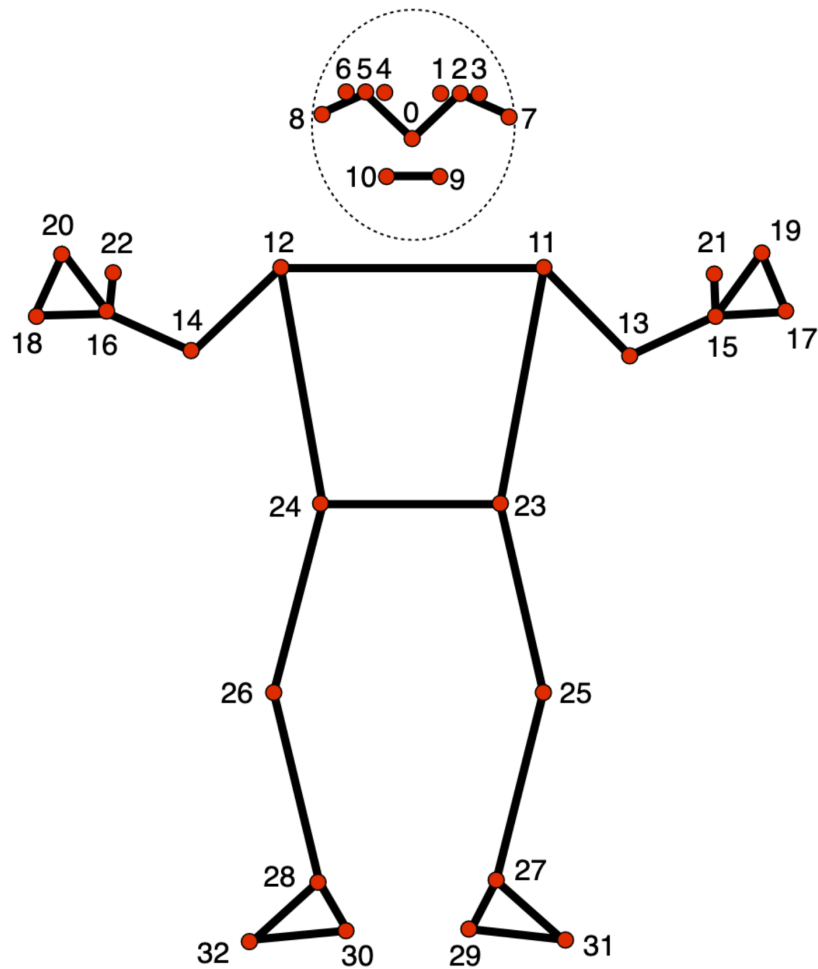


Figure 2.8: Mediapipe skeleton estimation example showing body landmarks [10]

2.3.3 PyKinect and the Kinect device

The Kinect device serves as a pivotal component in our system, enabling advanced motion sensing and skeletal tracking capabilities. Below is an overview of its main features:

Overview of the Kinect device

The Kinect device, as shown in Figure 2.10, developed by Microsoft, is a motion-sensing device that captures depth data and RGB images [13]. It was originally designed for use with gaming consoles but has since found applications in various fields, including healthcare and rehabilitation.

Key features of the Kinect device include:

- **Depth Sensing**

Uses infrared light to determine the distance of objects from the device, creating a depth map of the environment.

- **Skeletal Tracking**

Identifies and tracks 3D coordinates of 25 body joints. An example of this skeletal joints can be found in Figure 2.9, enabling detailed analysis of human movement.

- **RGB Camera**

Captures color images that can be used in conjunction with depth data.

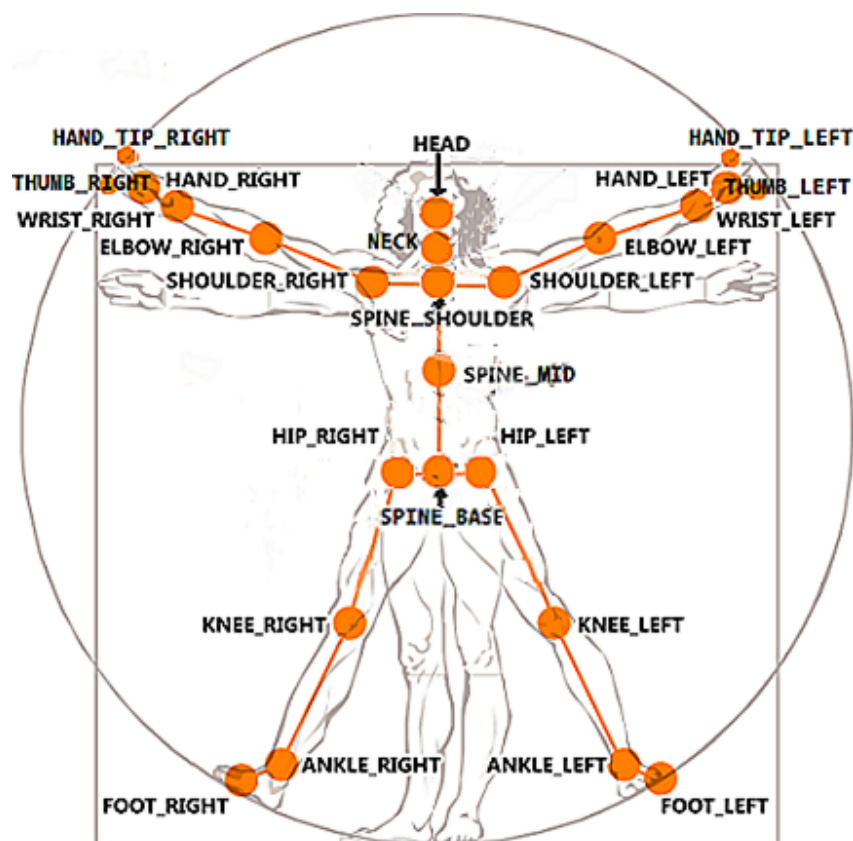


Figure 2.9: Microsoft Kinect skeleton estimation example showing body landmarks [13]

Appearance of the kinect device and location of the sensors

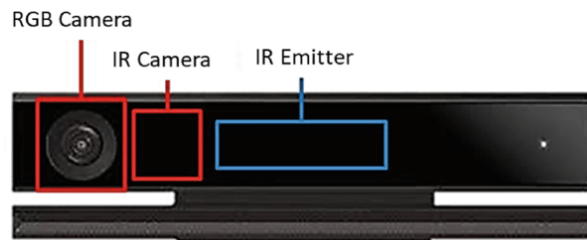


Figure 2.10: Microsoft Kinect device used for depth sensing and skeletal tracking [13]

PyKinect Library

PyKinect is a Python wrapper for the Kinect SDK that allows developers to access the Kinect's data streams and functionalities within Python applications.

Advantages of using PyKinect include:

- **Access to Depth Data**

Enables precise distance measurements by utilizing the depth information provided by the device.

- **Skeletal Tracking Capabilities**

Facilitates the detection and tracking of body joints in 3D space.

- **Integration with Python**

Allows for rapid development and integration with other Python libraries.

Relevance to This Project

By using PyKinect and the Kinect device, we can achieve:

- **High Precision Measurements**

Depth data enhances the accuracy of distance calculations between body landmarks.

- **Robustness in Various Conditions**

Infrared sensing is less affected by lighting conditions compared to RGB cameras.

- **3D Analysis**

Provides three-dimensional positional data, which can be valuable for comprehensive movement analysis.

2.4 System Requirements

This section delineates the essential system requirements necessary for the effective implementation and operation of the distance measurement system using computer vision methods. It encompasses both functional and technical specifications, detailing the required hardware components such as processors, memory, and sensors, as well as the necessary software dependencies and libraries for the Mediapipe and PyKinect implementations.

2.4.1 Functional Requirements

To fulfill the project's objectives, the system must meet the following functional requirements:

- **Accurate Body Landmark Detection**

The system must reliably detect and track relevant body landmarks necessary for the FB tests, such as fingertips, wrists, elbows, shoulders, hips, knees, and ankles.

- **Precise Distance Calculation**

It must compute the distances between specified landmarks with a high degree of accuracy to ensure valid assessment results.

- **Real-Time Processing Capability**

The system should process input data and provide measurements in real-time or near-real-time to facilitate efficient testing sessions.

2.4.2 Technical Requirements

Technical requirements include:

- **Hardware Compatibility**

- For the Mediapipe version, a standard computer with a webcam is required.
- For the PyKinect version, a Kinect device (e.g., Kinect for Windows v2) and compatible computer hardware are necessary.

- **Software Dependencies**

- **Mediapipe Version**

Requires Python 3.x, Mediapipe library, OpenCV for image processing, and NumPy for numerical computations.

- **PyKinect Version**

Requires Python 3.x, PyKinect library, Kinect SDK, and associated drivers.

- **Operating System**

The system should be compatible with common operating systems such as Windows 10/11 for the PyKinect version (due to Kinect SDK requirements) and cross-platform support for the Mediapipe version.

2.5 Specific Objectives

This section outlines the specific objectives that drive the development and implementation of the distance measurement system using computer vision methods. It details the precise goals aimed at achieving accurate and reliable measurements, including the creation of two distinct program versions leveraging Mediapipe and PyKinect technologies.

2.5.1 Objective 1: Develop Measurement Algorithms

Design and implement algorithms that can accurately measure distances between specific body landmarks required for the FB tests.

- **Chair Sit-and-Reach Test**

Measure the linear distance between the fingertips and the toes when the participant reaches forward from a seated position with one leg extended, as illustrated in Figure 2.11.



Figure 2.11: Chair Sit-and-Reach visual representation

- **Back Scratch Test**

Calculate the vertical or horizontal distance between the middle fingertips of both hands when the participant reaches one hand over the shoulder and the other up the middle of the back, as illustrated in Figure 2.12.



Figure 2.12: Back Scratch visual representation

2.5.2 Objective 2: Implement Two Program Versions

Develop two versions of the program to explore different technological approaches and compare their effectiveness.

- **Version 1: Mediapipe**

- Utilize Mediapipe's pose estimation to detect 2D body landmarks from RGB images.
- Implement algorithms to calculate distances based on pixel coordinates, considering camera calibration for accurate real-world measurements.

- **Version 2: PyKinect**

- Use the Kinect sensor's depth data and skeletal tracking to obtain 3D coordinates of body joints.
- Calculate distances in three-dimensional space, leveraging depth information for enhanced accuracy.

2.5.3 Objective 3: Validate and Test the Programs

Conduct comprehensive testing to validate the accuracy, reliability, and robustness of both program versions.

- **Accuracy Assessment**

Compare the automated measurements with manual measurements obtained using traditional methods (e.g., measuring tape) to evaluate precision.

- **Performance Evaluation**

Assess the system's performance under different conditions, including varying lighting, backgrounds, and participant clothing.

- **Participant Diversity**

Test the program with participants of different ages, body types, and mobility levels to ensure generalizability.

- **Feedback Collection**

Gather feedback from participants and professionals to identify areas for improvement and assess usability.

With a thorough understanding of the background, technologies, and objectives, we proceed to the system architecture and setup details in the next chapter.

Chapter 3

System Architecture and Setup Details

This chapter provides a comprehensive overview of the system architecture and the setup details necessary for the accurate measurement of distances between human body segments using computer vision methods. It covers the hardware and software components, environmental considerations, user positioning, and includes detailed explanations of the code implementations for both the Mediapipe and PyKinect versions. Additionally, it explains how using the Kinect sensor for both implementations ensures consistent and accurate comparison results.

3.1 System Architecture

The system is designed to capture and process human body movements to measure distances between specific body landmarks accurately. To obtain consistent and comparable results between the two implementations, both the Mediapipe and PyKinect versions utilize the Kinect device as the camera input device. This approach ensures that the same hardware conditions are maintained, allowing for a fair comparison of the software methodologies.

3.1.1 Hardware Components

The effectiveness of the distance measurement system is largely dependent on the quality and capabilities of its hardware components. The following key devices are integral to capturing and processing the necessary data for accurate pose estimation and analysis:

- **Camera Sensor – Kinect Sensor**

The Microsoft Kinect device (e.g., Kinect for Windows v2) is used as the camera input device for both implementations. The Kinect device provides:

- High-quality RGB camera for capturing color images.
- Depth sensor for obtaining 3D data of the environment.
- Infrared sensor for enhanced skeletal recognition, especially in low-light conditions.
- Wide field of view to capture the full body of the user.

- **Computer System**

The system requires a computer with the following specifications:

- Multi-core processor (Intel Core i5 or higher) for handling real-time data processing.
- At least 8 GB of RAM to manage computational tasks efficiently.
- USB 3.0 port to connect the Kinect device, ensuring high data transfer rates.
- Graphics processing unit (GPU) is beneficial but not mandatory. A dedicated GPU can accelerate computations, especially when processing high-resolution images.
- Operating System: Windows 10 or higher for full compatibility with the Kinect SDK.

3.1.2 Software Components

The system utilizes a combination of programming languages and specialized libraries/frameworks to enable accurate pose estimation and efficient data processing. The key software elements are detailed below:

- **Programming Language**

- Python 3.8 is utilized for both versions due to its extensive library support, ease of use, and strong community support. Python’s simplicity allows for rapid development and testing.

- **Libraries and Frameworks**

- **OpenCV:** An open-source computer vision and machine learning software library used for image and video processing tasks, such as reading frames from the Kinect’s RGB camera and displaying results.
- **Mediapipe:** A cross-platform framework by Google that provides customizable machine learning solutions for live and streaming media. Used for pose estimation in the Mediapipe

version.

- **PyKinect2:** A Python wrapper for the Kinect for Windows SDK, allowing access to the Kinect’s depth data, skeletal tracking, and RGB camera frames.
- **NumPy:** A fundamental package for scientific computing with Python, providing support for arrays and matrices, along with a collection of mathematical functions.

3.1.3 System Workflow

The system follows a series of steps, as provided in Figure 3.1, to perform the measurements:

1. Data Acquisition

- Capturing real-time video frames from the Kinect’s RGB camera for both implementations. For the PyKinect version, depth data and skeletal tracking information are also acquired.

2. Pre-processing

- Resizing, normalizing, and preparing frames for analysis to optimize performance and accuracy. This may include converting images to different color spaces or applying filters to enhance image quality.

3. Pose Estimation

- **Mediapipe Version:** Uses Mediapipe’s pose estimation on RGB images captured by the Kinect. Mediapipe detects 33 key body landmarks in 2D space.
- **PyKinect Version:** Utilizes the Kinect’s depth data and skeletal tracking capabilities to obtain 3D coordinates of 25 body joints.

4. Feature Extraction

- Extracting relevant positional data (coordinates) from the detected landmarks or joints. For instance, extracting the positions of the hand, foot, knee, hip, and other relevant joints.

5. Distance and Angle Calculation

- Computing distances between specific body landmarks using Euclidean distance formulas in 2D (Mediapipe) or 3D (PyKinect) space.
- Calculating joint angles using the law of cosines or vector mathematics to assess posture correctness and movement quality.

6. Flexibility Scoring

- Evaluating the user's performance by calculating a flexibility score based on the measured distances and angles. The score takes into account factors such as how close the user can reach to their toes and the angles of key joints.

7. Feedback Visualization

- Displaying visual feedback, including progress bars, flexibility bars, joint angle visualizations, and annotations, to guide the user in real-time. This helps users understand their performance and make adjustments as needed.

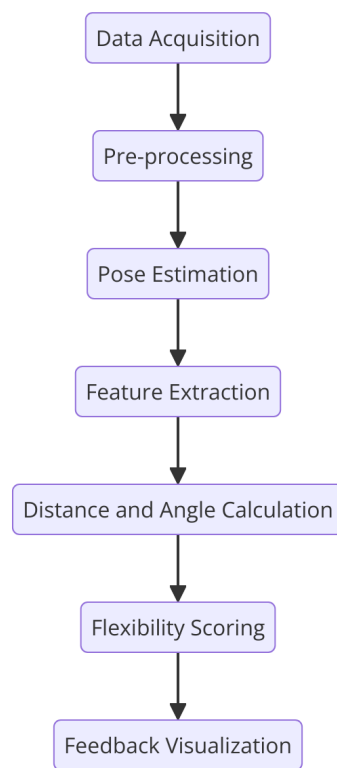


Figure 3.1: System architecture workflow diagram illustrating the sequence of processes from data acquisition to feedback visualization

3.1.4 Data Flow and Processing

The data flow and processing steps are crucial for understanding how raw data is transformed into meaningful feedback:

- **Frame Capture:** The Kinect device captures frames at a resolution of 1920x1080 pixels for the RGB camera and 512x424 pixels for depth data.

- **Data Synchronization:** Ensuring that color and depth frames are synchronized is important, especially when combining data from both streams.
- **Coordinate Mapping:** In the PyKinect version, mapping the 3D skeletal data to the 2D color frame coordinates is necessary for visualization.
- **Error Handling:** Implementing checks for tracking states and visibility of joints to handle cases where certain landmarks are not detected.
- **Performance Optimization:** Techniques such as multi-threading or utilizing GPU acceleration can improve the system's responsiveness and reduce latency.

3.2 Setup Details

This section outlines the detailed setup procedures essential for implementing the distance measurement system using computer vision methods. It encompasses the configuration and installation of both hardware and software components, including the calibration of sensors, setup of computing resources, and installation of necessary software environments and dependencies.

3.2.1 Software Environment Setup

To set up the software environment, follow these detailed instructions.

Installing Anaconda

Anaconda is a free and open-source distribution of Python that includes a package manager (conda) and a collection of pre-installed packages. It simplifies package management and deployment.

1. Download Anaconda

Visit the official Anaconda website and download the latest version of Anaconda for Windows (Python 3.x version) [18].

2. Install Anaconda

Run the downloaded installer and follow the on-screen instructions:

- Agree to the license agreement.
- Choose the installation type (Just Me).
- Select the installation location (default is recommended).

- During the installation process, you may be prompted to add Anaconda to your PATH environment variable. It is recommended to leave this option unchecked to avoid conflicts with other Python installations.

Creating a Virtual Environment

Creating a virtual environment helps in managing project-specific dependencies without affecting other projects.

1. Open Anaconda Prompt

Go to the Start Menu and search for "Anaconda Prompt". Open it.

2. Create a New Environment

In the Anaconda Prompt, run the following command to create a new environment named `capcita_env` (you can choose any name):

```
conda create -n capcita_env python=3.8
```

3. Activate the Environment

Activate the newly created environment:

```
conda activate capcita_env
```

Installing Required Libraries

With the environment activated, install the necessary libraries.

1. Install OpenCV

```
conda install -c conda-forge opencv
```

2. Install Mediapipe

```
pip install mediapipe
```

3. Install NumPy

```
conda install numpy
```

4. Install PyKinect2

PyKinect2 is not available via conda or pip directly. You need to install it manually.

- Download the PyKinect2 package from the GitHub repository. [\[5\]](#)
- Alternatively, you can clone the repository using Git:

```
git clone https://github.com/Kinect/PyKinect2.git
```

- Navigate to the downloaded folder:

```
cd PyKinect2
```

- Install the package:

```
python setup.py install
```

5. Install Additional Packages

Install any additional packages that may be required:

```
conda install matplotlib
```

```
conda install -c anaconda scipy
```

Installing Kinect for Windows SDK

The Kinect for Windows SDK is required to use the Kinect device with your computer.

1. Download the Kinect SDK

Visit and download the Kinect for Windows SDK 2.0.[\[19\]](#)

2. Install the SDK

Run the installer and follow the on-screen instructions to complete the installation.

3. Verify Installation

After installation, you can verify that the Kinect device is working by running the Kinect Studio application included with the SDK.

Setting Up Spyder IDE

Spyder is included with Anaconda but may need to be installed in your new environment.

1. Install Spyder in the Environment

With your environment activated, install Spyder:

```
conda install spyder
```

2. Launch Spyder

Start Spyder from the Anaconda Prompt:

```
spyder
```

Alternatively, you can launch Spyder from the Start Menu. Ensure that you select the correct environment when Spyder starts.

3. Configure Spyder

Within Spyder, you can configure settings such as working directory, syntax highlighting, and keyboard shortcuts according to your preferences.

Testing the Setup

To verify that everything is installed correctly:

1. Create a Test Script

In Spyder, create a new Python script and add the code provided in Listing 3.1, to test the imports:

```
1  import cv2
2  import mediapipe as mp
3  import numpy as np
4  from pykinect2 import PyKinectRuntime, PyKinectV2
5
6  print("All libraries imported successfully.")
```

Listing 3.1: Test script to verify imports

2. Run the Script

Execute the script. If no errors occur and the message "All libraries imported successfully." is printed, the libraries are installed correctly.

3. Test Kinect Sensor Connection

Add the following code, provided in Listing 3.2, to initialize the Kinect device:

```
1    kinect = PyKinectRuntime.PyKinectRuntime(PyKinectV2.  
        FrameSourceTypes_Color)  
2    if kinect:  
3        print("Kinect device initialized successfully.")  
4    else:  
5        print("Failed to initialize Kinect device.")
```

Listing 3.2: Test script to verify Kinect device

Run the script again. If the Kinect device is initialized successfully, your setup is complete.

3.2.2 Hardware Setup

This section details the configuration and assembly of the hardware components required for the distance measurement system.

Connecting the Kinect Sensor

1. Power Connection

Connect the Kinect device to the power supply using the provided power adapter.

2. USB Connection

Plug the Kinect device's USB cable into a USB 3.0 port on your computer. USB 3.0 ports are typically marked with a blue color or the SS (SuperSpeed) logo.

3. Device Recognition

Wait for Windows to recognize the device. You may see a notification indicating that the Kinect device is connected and ready to use.

4. Driver Installation

If prompted, allow Windows to install the necessary drivers. This should happen automatically if the Kinect SDK is installed.

3.2.3 Running the Code

With the setup complete, you can now run the code implementations.

Mediapipe Implementation

1. **Open the Mediapipe Script**

In Spyder, open the Python script containing the Mediapipe implementation code.

2. **Run the Script**

Execute the script by clicking the Run button or pressing F5.

3. **Allow Access to Camera**

If prompted, allow the script to access the camera.

4. **Perform the Exercise**

Follow the on-screen instructions to perform the chair sit-and-reach exercise.

5. **View Results**

The script will display real-time feedback and, upon completion, show the final results.

PyKinect Implementation

1. **Open the PyKinect Script**

In Spyder, open the Python script containing the PyKinect implementation code.

2. **Run the Script**

Execute the script.

3. **Perform the Exercise**

Follow the instructions to perform the exercise while the Kinect device captures your movements.

4. **View Results**

The script will provide real-time feedback and display the final results upon completion.

3.2.4 Troubleshooting Common Issues

This section outlines common problems encountered during the setup and operation of the distance measurement system and provides practical solutions to resolve them.

- **Library Import Errors**

Ensure that you have installed all required libraries in the active conda environment. If you receive an import error, double-check the installation commands and try reinstalling the library.

- **Kinect Sensor Not Detected**

Verify that the Kinect device is properly connected to a USB 3.0 port and that the power supply is connected. Check the Device Manager to see if the Kinect is recognized.

- **Permission Issues**

If you encounter permission errors when accessing the camera or device, run Spyder or the Anaconda Prompt as an administrator.

- **Version Compatibility**

Ensure that you are using compatible versions of Python and the libraries. Some libraries may have specific version requirements.

- **Performance Issues**

Close unnecessary applications to free up system resources. If the system is still slow, consider upgrading your hardware or optimizing the code.

Positioning the Kinect Sensor

Proper setup of the system is crucial to ensure accurate motion tracking and measurement. This section details the environmental setup, device positioning, and user positioning guidelines.

3.2.5 Environmental Setup

This section outlines the necessary environmental conditions and configurations required to ensure the accurate and reliable operation of the distance measurement system.

Location

The system should be set up in a well-lit room with minimal obstructions. Adequate lighting ensures that the Kinect's RGB camera can capture clear images, which is essential for accurate pose estimation using Mediapipe. Avoid direct sunlight, reflective surfaces, or harsh shadows that could interfere with the device's ability to detect body landmarks.

Background

A plain, uncluttered background is preferable. Busy or dynamic backgrounds can confuse the pose estimation algorithms, leading to decreased accuracy. Using a solid-colored backdrop or a wall can enhance detection reliability.

Movement Area

A clear area of approximately 2 meters by 2 meters around the user should be maintained. This space allows the user to perform the exercises freely without any obstructions that could interfere with motion tracking or pose estimation.

Noise and Interference

Minimize electronic interference from other devices that might affect the Kinect device's performance. Ensure that there are no reflective or shiny surfaces that could distort depth sensing.

3.2.6 Device Positioning

Below provided instructions for device positioning, with illustration in Figure 3.2:

Mounting Location

The Kinect device should be placed below the display screen, centrally aligned with the user. This positioning ensures that the device's field of view encompasses the user's entire body from head to toe.

Height from Ground

The device should be positioned at a height of approximately 45 to 60 centimeters from the ground. This height allows the device to capture lower body movements effectively, which is critical for exercises involving the legs and feet.

Angle

The device should be angled upward at approximately 10 to 15 degrees. This slight upward tilt helps prevent occlusions and ensures that the entire body, including the feet and hands, is within the device's field of view.

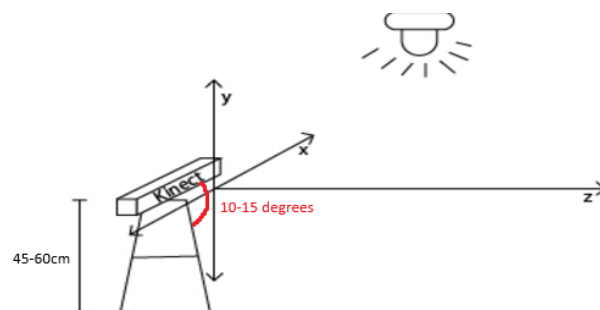


Figure 3.2: Optimal Kinect device positioning and alignment for accurate motion capture

Orientation

Proper device orientation is essential for accurate tracking. The following factors should be considered:

- **Alignment:** The device must be centrally aligned with the user's position to ensure symmetrical tracking and minimize distortions.
- **Adjustability:** The mounting setup should allow for minor adjustments to fine-tune the device's angle and position, optimizing tracking accuracy.
- **Stability:** Ensure that the device is securely mounted to prevent vibrations or movements that could disrupt data capture.
- **Field of View:** Verify that the device's field of view covers the entire area where the user will perform the exercises. The Kinect v2 has a horizontal field of view of 70 degrees and a vertical field of view of 60 degrees.
- **Calibration:** Regular calibration is necessary to account for any shifts in position and ensure continued optimal performance. Calibration involves adjusting the device settings and verifying that the captured data accurately reflects the user's movements.

3.2.7 User Positioning

Below provided instruction for user positioning, with illustration in [Figure 3.3](#):

Distance from device

Users should stand at a distance of 2.5 to 3.5 meters from the device. This range is optimal for accurate motion tracking without exceeding the device's depth limitations. Standing too close may result in parts of the body being out of frame, while standing too far may reduce the resolution of the captured data.

Central Alignment

Users should be aligned centrally in front of the device. This alignment ensures that their entire body is captured within the device's field of view and that movements are tracked symmetrically. Markers on the floor can help users maintain the correct position.

Posture and Movement

Users should maintain a natural posture and avoid excessive movements outside the designated exercise. Smooth and deliberate movements improve tracking accuracy and reduce the likelihood of errors in

measurement. Instructions should be provided to the user to ensure they understand the required movements.

Clothing and Accessories

Participants should wear form-fitting clothing that does not obscure body contours. Loose or baggy clothing can hinder the detection of body landmarks. Additionally, removing accessories like hats, watches, or jewelry that could occlude key points is recommended.

Footwear

Wearing flat, non-reflective footwear is advisable. Reflective surfaces on shoes can interfere with depth sensing and landmark detection.

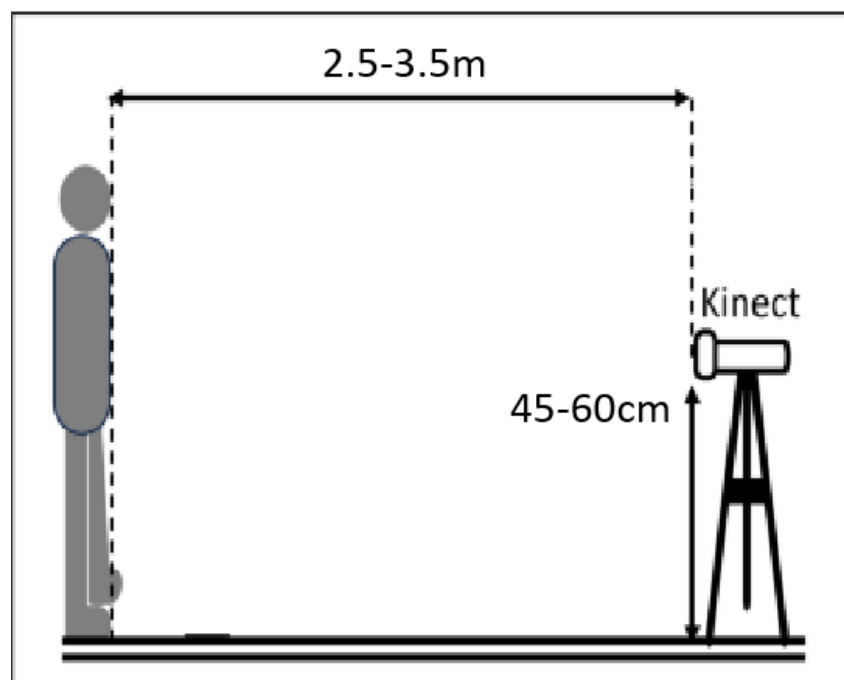


Figure 3.3: User positioning relative to the device, ensuring full-body capture and optimal distance

3.3 Code Implementations

This section provides a detailed explanation of the code implementations for both the Mediapipe and PyKinect versions. The code is responsible for capturing data, performing pose estimation, calculating distances and angles, scoring flexibility, and providing visual feedback. By understanding the code in depth, one can appreciate the intricacies involved in measuring distances between human body segments using computer vision methods.

3.3.1 Mediapipe Version with Kinect Input

The Mediapipe version utilizes the Kinect's RGB camera as the input device for pose estimation. Mediapipe's pose detection solution processes the RGB images to identify and track body landmarks. The code is structured to capture frames, process them through Mediapipe, perform necessary calculations, and provide real-time feedback to the user.

Code Explanation

In this section, we will dissect the code step by step, explaining each component in detail.

Importing Libraries

The first step involves importing the necessary libraries required for the program, as provided in Listing 3.3. These libraries provide functionalities for image processing, pose estimation, numerical computations, and accessing the Kinect device.

```
1 import cv2
2 import mediapipe as mp
3 import numpy as np
4 import time
5 from pykinect2 import PyKinectRuntime, PyKinectV2
```

Listing 3.3: Importing required libraries

- **cv2**: OpenCV library for computer vision tasks, such as image and video processing.
- **mediapipe**: Provides the pose estimation solution used to detect and track body landmarks.
- **numpy**: Used for numerical computations and array manipulations.
- **time**: Allows tracking of time intervals, which is useful for measuring how long the user holds a pose.
- **pykinect2**: A Python wrapper for the Kinect SDK, enabling access to the Kinect's color frames.

Initializing Kinect and Mediapipe Pose

We initialize the Kinect device to capture color frames and set up the Mediapipe pose estimation model, as provided in Listing 3.4.

```
1 # Initialize Kinect
2 kinect = PyKinectRuntime(PyKinectV2.FrameSourceTypes_Color)
```

```
3
4 # Initialize Mediapipe Pose
5 mp_pose = mp.solutions.pose
6 pose = mp_pose.Pose()
7 mp_drawing = mp.solutions.drawing_utils
```

Listing 3.4: Initializing Kinect and Mediapipe Pose

- `PyKinectRuntime.PyKinectRuntime(PyKinectV2.FrameSourceTypes_Color)`: Initializes the Kinect device to capture color frames.
- `mp_pose.Pose()`: Creates an instance of the Mediapipe pose estimation model.
- `mp_drawing`: Provides utilities for drawing the pose landmarks on the image.

Function Definitions

Several helper functions are defined to perform calculations and visualizations:

1. Calculating Euclidean Distance in 2D

This function shown in Listing 3.5 computes the Euclidean distance between two points in 2D space, which is essential for measuring the distance between body landmarks.

```
1 def calculate_distance_2d(point1, point2):
2     return np.sqrt((point1[0] - point2[0])**2 + (point1[1] - point2[1])**2)
```

Listing 3.5: Function to calculate Euclidean distance in 2D

2. Adjusting Fingertip Position

This function shown in Listing 3.6 adjusts the position of the fingertip to account for measurement inaccuracies or to simulate a more precise fingertip position.

```
1 def adjust_fingertip_position(index, adjustment=0.02):
2     return np.array([index.x, index.y - adjustment]) # Move 2 cm down (y-
axis)
```

Listing 3.6: Function to adjust fingertip position

3. Calculating Angle Between Three Points

This function shown in Listing 3.7 calculates the angle at point `b` formed by points `a`, `b`, and `c`. It uses the arctangent of the slopes between the points.

```
1 def calculate_angle(a, b, c):
2     a = np.array(a) # First point
3     b = np.array(b) # Mid point
4     c = np.array(c) # End point
5
6     radians = np.arctan2(c[1] - b[1], c[0] - b[0]) - \
7                 np.arctan2(a[1] - b[1], a[0] - b[0])
8     angle = np.abs(radians * 180.0 / np.pi)
9
10    if angle > 180.0:
11        angle = 360 - angle
12
13    return angle
```

Listing 3.7: Function to calculate angle between three points

4. Calculating Flexibility Score

This function shown in Listing 3.8 computes a flexibility score based on the distance and joint angles. It normalizes the measurements and applies weights to emphasize certain aspects.

```
1 def calculate_flexibility_score(distance, knee_angle, hip_angle,
2     elbow_angle, max_distance=0.7):
3     # Normalize the distance score (0 to 1)
4     normalized_distance = (max_distance - distance) / max_distance
5     normalized_distance = max(0.0, min(normalized_distance, 1.0)) # Clamp
6     # Normalize angles
7     normalized_knee = (knee_angle - 140) / (180 - 140)
8     normalized_hip = (hip_angle - 50) / (130 - 50)
9     normalized_elbow = (elbow_angle - 150) / (180 - 150)
10
11    # Adjust weights
12    flexibility_score = (normalized_distance * 0.55) + \
13                        (normalized_knee * 0.25) + \
14                        (normalized_hip * 0.15) + \
15                        (normalized_elbow * 0.05)
16    return max(0.0, min(flexibility_score, 1.0))
```

Listing 3.8: Function to calculate flexibility score

5. Classifying Flexibility Score

This function shown in Listing 3.9 in Listing categorizes the flexibility score into 'Bad', 'Good', or 'Nice' based on predefined thresholds .

```
1 def classify_flexibility_score(flexibility_score):
2     if flexibility_score < 0.6:
3         return "Bad"
4     elif 0.6 <= flexibility_score < 0.85:
5         return "Good"
6     else:
7         return "Nice"
```

Listing 3.9: Function to classify flexibility score

6. Drawing Dynamic Angle Arc

This function shown in Listing 3.10 visualizes joint angles by drawing arcs on the listing.

```
1 def draw_dynamic_angle_arc(image, p1, p2, p3, angle):
2     # Draw lines between points
3     cv2.line(image, p1, p2, (255, 255, 0), 2)
4     cv2.line(image, p2, p3, (255, 255, 0), 2)
5
6     # Calculate the arc center point
7     arc_center = tuple(p2.astype(int))
8
9     # Calculate radius
10    radius = int(np.linalg.norm(np.array(p1) - np.array(p2)) / 2)
11
12    # Draw filled arc
13    axes = (radius, radius)
14    cv2.ellipse(image, arc_center, axes, 0, 0, angle, (0, 255, 0), -1)
15    cv2.ellipse(image, arc_center, axes, 0, 0, angle, (0, 0, 255), 2)
```

Listing 3.10: Function to draw dynamic angle arc

7. Drawing Progress Bar

This function shown in Listing 3.11 draws a progress bar indicating how long the correct pose has been held.

```
1 def draw_progress_bar(image, progress, position, size=(200, 20)):
2     cv2.rectangle(image, position, (position[0] + size[0], position[1] +
3         size[1]),
4         (255, 255, 255), 1)
5     cv2.rectangle(image, position,
6         (position[0] + int(size[0] * progress), position[1] +
7             size[1]),
8         (0, 255, 0), -1)
```

Listing 3.11: Function to draw progress bar

8. Drawing Flexibility Bar

This function shown in Listing 3.12 displays a bar indicating the flexibility score, divided into segments representing different performance levels.

```
1 def draw_flexibility_bar(image, flexibility_score, position=(50, 400), size
2     =(300, 50)):
3     # Define thresholds
4     bad_threshold = 0.6
5     good_threshold = 0.25
6     nice_threshold = 0.15
7
8     # Draw the bar background
9     cv2.rectangle(image, position, (position[0] + size[0], position[1] +
10         size[1]),
11         (255, 255, 255), 2)
12
13     # Calculate widths
14     bad_width = int(size[0] * bad_threshold)
15     good_width = int(size[0] * good_threshold)
16     nice_width = size[0] - bad_width - good_width
```

Listing 3.12: Part of function to draw flexibility bar

9. Calculating Average Distance

This function shown in Listing 3.13 calculates the average of the last few distance measurements to smooth out fluctuations.

```
1 def average_distance(distances):  
2     return sum(distances) / len(distances)
```

Listing 3.13: Function to calculate average distance

Main Loop

The main loop shown in Listing 3.14 is responsible for capturing frames, processing them, performing calculations, and displaying the results. It runs continuously until the user decides to exit.

```
1 # Variables for tracking  
2 distances = []  
3 average_over = 5  
4 final_distance = None  
5 pose_correct_start_time = None  
6 pose_held_duration = 8 # seconds  
7  
8 # Main loop to capture and process frames  
9 while True:  
10     if kinect.has_new_color_frame():  
11         # Get the latest color frame  
12         frame = kinect.get_last_color_frame()  
13         frame = frame.reshape((1080, 1920, 4)) # Kinect frame dimensions  
14         rgb_image = cv2.cvtColor(frame, cv2.COLOR_BGRA2BGR) # Convert to BGR  
15  
16         # Convert to RGB for Mediapipe  
17         rgb_frame = cv2.cvtColor(rgb_image, cv2.COLOR_BGR2RGB)  
18         rgb_frame.flags.writeable = False  
19  
20         # Process with Mediapipe  
21         results = pose.process(rgb_frame)  
22  
23         # Recolor back to BGR  
24         rgb_frame.flags.writeable = True  
25         frame = cv2.cvtColor(rgb_frame, cv2.COLOR_RGB2BGR)  
26
```

```
27 # Rest of the processing...
```

Listing 3.14: Main loop for frame processing

Pose Estimation and Calculations

After processing the frame with Mediapipe, we check if any pose landmarks are detected and proceed with calculations, as shown in Listing 3.15.

```
1 # Check if pose landmarks are detected
2 if results.pose_landmarks:
3     landmarks = results.pose_landmarks.landmark
4
5     # Ensure required landmarks are visible
6     required_landmarks = [
7         mp_pose.PoseLandmark.RIGHT_WRIST,
8         mp_pose.PoseLandmark.RIGHT_INDEX,
9         mp_pose.PoseLandmark.RIGHT_FOOT_INDEX,
10        mp_pose.PoseLandmark.RIGHT_HIP,
11        mp_pose.PoseLandmark.RIGHT_KNEE,
12        mp_pose.PoseLandmark.RIGHT_ANKLE,
13        mp_pose.PoseLandmark.RIGHT_SHOULDER,
14        mp_pose.PoseLandmark.RIGHT_ELBOW
15    ]
16
17    if all(landmarks[lm.value].visibility > 0.5 for lm in required_landmarks):
18        # Extract necessary landmarks
19        right_wrist = landmarks[mp_pose.PoseLandmark.RIGHT_WRIST.value]
20        right_index = landmarks[mp_pose.PoseLandmark.RIGHT_INDEX.value]
21        right_foot = landmarks[mp_pose.PoseLandmark.RIGHT_FOOT_INDEX.value]
```

Listing 3.15: Part of pose estimation and calculations

Visualization

We provide real-time feedback to the user by drawing on the frame, as shown in Listing 3.16:

```
1 # Draw landmarks and connections
2 mp_drawing.draw_landmarks(frame, results.pose_landmarks, mp_pose.
    POSE_CONNECTIONS,
```

```

3             mp_drawing.DrawingSpec(color=(245, 117, 66),
4                                     thickness=2, circle_radius=2),
5
6             mp_drawing.DrawingSpec(color=(245, 66, 230),
7                                     thickness=2, circle_radius=2))
8
9
10 # Display distance
11 cv2.putText(frame, f'Distance: {distance:.2f} m', (10, 110),
12             cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
13
14 # Visualize knee angle
15 knee_coords = tuple(np.multiply(right_knee, [frame.shape[1], frame.shape[0]]).
16                     astype(int))
17 hip_coords = tuple(np.multiply(right_hip, [frame.shape[1], frame.shape[0]]).
18                    astype(int))
19 ankle_coords = tuple(np.multiply(right_ankle, [frame.shape[1], frame.shape[0]]).
20                      astype(int))
21 draw_dynamic_angle_arc(frame, hip_coords, knee_coords, ankle_coords, knee_angle
22                        )
23 cv2.putText(frame, f'Knee Angle: {knee_angle:.2f}', knee_coords,
24            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

```

Listing 3.16: Part of visualization and feedback

Final Result Visualization

Once the exercise is completed, we display the final results, as shown in Listing 3.17:

```

1 if final_distance is not None:
2     final_frame = np.zeros((500, 800, 3), dtype=np.uint8) # Black screen for
3     final results
4
5     flexibility_result = classify_flexibility_score(flexibility_score)
6
7     cv2.putText(final_frame, 'Exercise Completed', (200, 100),
8                 cv2.FONT_HERSHEY_SIMPLEX, 1.5, (255, 255, 255), 2)
9     cv2.putText(final_frame, f'Final Distance: {final_distance:.2f} meters',
10                (100, 200),
11                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
12     cv2.putText(final_frame, f'Flexibility Result: {flexibility_result} ({
13         flexibility_score:.2f})',

```

```

10         (100, 300), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
11
12     cv2.imshow('Final Results', final_frame)
13     cv2.waitKey(0)

```

Listing 3.17: Displaying final results

Algorithm Flowchart

To provide a visual understanding of the code flow, refer to Figure 3.4, which illustrates the steps involved in the Mediapipe implementation:

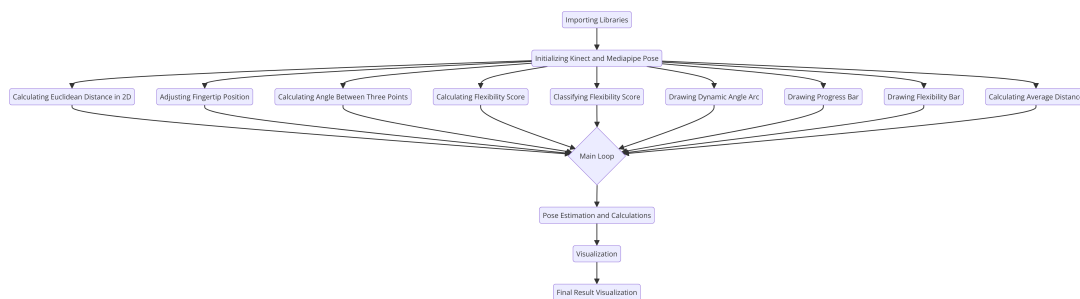


Figure 3.4: Flowchart of the Mediapipe implementation using Kinect camera

- **Importing Libraries:** Loads the necessary Python libraries and frameworks required for processing and analysis.
- **Initializing Kinect and Mediapipe Pose:** Sets up and configures the Kinect sensor alongside Mediapipe for real-time pose detection.
- **Calculating Euclidean Distance in 2D:** Measures the straight-line distance between two points in a two-dimensional plane.
- **Adjusting Fingertip Position:** Refines the position data of fingertips to enhance the accuracy of measurements.
- **Calculating Angle Between Three Points:** Determines the angle formed by three landmarks to assess joint flexibility.
- **Calculating Flexibility Score:** Computes an overall flexibility score based on distance and angle measurements.
- **Classifying Flexibility Score:** Categorizes the flexibility score into predefined classes such as Bad, Good, or Excellent.

- **Drawing Dynamic Angle Arc:** Visualizes the calculated angles on the user's skeletal model for real-time feedback.
- **Drawing Progress Bar:** Displays a progress bar to indicate the completion status of the assessment process.
- **Drawing Flexibility Bar:** Shows a bar representing the user's flexibility score for easy interpretation.
- **Calculating Average Distance:** Averages multiple distance measurements to improve the reliability of the data.
- **Main Loop:** Continuously runs the core processes, handling data capture, processing, and visualization in real-time.
- **Pose Estimation and Calculations:** Estimates the user's pose and performs necessary calculations for flexibility assessment.
- **Visualization:** Provides real-time visual feedback based on the pose estimations and calculated scores.
- **Final Result Visualization:** Presents the final flexibility assessment results to the user in an understandable format.

3.3.2 PyKinect Version

The PyKinect version leverages the Kinect device's depth data and skeletal tracking capabilities to obtain 3D coordinates of body joints. This implementation provides a more direct access to skeletal data, which can be advantageous for certain applications.

Code Explanation

We will now dissect the PyKinect code, explaining each component in detail.

Importing Libraries

The required libraries are imported, including those for accessing the Kinect device, image processing, and numerical computations, as shown in Listing 3.18.

```
1 import cv2
2 import numpy as np
3 from pykinect2 import PyKinectRuntime, PyKinectV2
4 import time
```

Listing 3.18: Importing required libraries

Initializing Kinect device

The Kinect device is initialized to capture color, body, depth, and infrared frames, as shown in Listing 3.19.

```
1 kinect = PyKinectRuntime.PyKinectRuntime(  
2     PyKinectV2.FrameSourceTypes_Color |  
3     PyKinectV2.FrameSourceTypes_Body |  
4     PyKinectV2.FrameSourceTypes_Depth |  
5     PyKinectV2.FrameSourceTypes_Infrared)
```

Listing 3.19: Initializing Kinect device

Function Definitions

Helper functions are defined for distance calculations, angle measurements, flexibility scoring, and visualization.

1. Calculating Euclidean Distance in 3D

The function below computes the Euclidean distance between two points in three-dimensional space. It takes two points as input and returns the straight-line distance separating them, as shown in Listing 3.20.

```
1 def calculate_distance_3d(point1, point2):  
2     return np.sqrt((point1[0] - point2[0]) ** 2 +  
3                     (point1[1] - point2[1]) ** 2 +  
4                     (point1[2] - point2[2]) ** 2)
```

Listing 3.20: Function to calculate Euclidean distance in 3D

2. Calculating Angle Between Three Points in 3D

The function below calculates the angle formed at point b by three points in three-dimensional space. It takes three points as input and returns the angle in degrees, as shown in Listing 3.21.

```
1 def calculate_angle(a, b, c):  
2     a = np.array(a)  
3     b = np.array(b)
```



```
4     c = np.array(c)
5
6     ab = a - b
7     cb = c - b
8
9     # Calculate cosine of angle using dot product
10    cosine_angle = np.dot(ab, cb) / (np.linalg.norm(ab) * np.linalg.norm(cb
11                                   ))
12    angle = np.arccos(cosine_angle) # Get angle in radians
13    return np.degrees(angle) # Convert to degrees
```

Listing 3.21: Function to calculate angle between three points in 3D

3. Drawing Progress Bar

The function below creates a visual progress bar on an image to represent the completion status of a task, as shown in Listing 3.22.

```
1 def draw_progress_bar(image, progress, position, size=(200, 20)):
2     cv2.rectangle(image, position, (position[0] + size[0], position[1] +
3                                     size[1]),
4                                     (255, 255, 255), 1)
5     cv2.rectangle(image, position,
6                   (position[0] + int(size[0] * progress), position[1] +
7                     size[1]),
8                   (0, 255, 0), -1)
```

Listing 3.22: Function to draw progress bar

4. Drawing Flexibility Bar

The function below creates a visual flexibility bar on an image, segmenting it into different performance levels based on predefined thresholds, as shown in Listing 3.23.

```
1 def draw_flexibility_bar(image, flexibility_score, position=(50, 400), size
2   =(300, 50)):
3     # Define thresholds
4     bad_threshold = 0.6
5     good_threshold = 0.25
6     nice_threshold = 0.15
```

```

7      # Draw the bar background
8      cv2.rectangle(image, position, (position[0] + size[0], position[1] +
          size[1]),
9                      (255, 255, 255), 2)
10     # Calculate widths
11     bad_width = int(size[0] * bad_threshold)
12     good_width = int(size[0] * good_threshold)
13     nice_width = size[0] - bad_width - good_width

```

Listing 3.23: Part of function to draw flexibility bar

5. Classifying Flexibility Score

The function below categorizes the flexibility score into predefined classes: "Bad", "Good", or "Nice", as shown in Listing 3.24.

```

1 def classify_flexibility_score(flexibility_score):
2     if flexibility_score < 0.6:
3         return "Bad"
4     elif 0.6 <= flexibility_score < 0.85:
5         return "Good"
6     else:
7         return "Nice"

```

Listing 3.24: Function to classify flexibility score

6. Calculating Flexibility Score

The function below computes an overall flexibility score by integrating measurements of distance and joint angles, as shown in Listing 3.25.

```

1 def calculate_flexibility_score(distance, knee_angle, hip_angle,
    elbow_angle, max_distance=0.7):
2     normalized_distance = (max_distance - distance) / max_distance
3     normalized_distance = max(0.0, min(normalized_distance, 1.0))
4
5     normalized_knee = (knee_angle - 140) / (180 - 140)
6     normalized_hip = (hip_angle - 50) / (130 - 50)
7     normalized_elbow = (elbow_angle - 150) / (180 - 150)
8
9     flexibility_score = (normalized_distance * 0.55) + \

```

```
10         (normalized_knee * 0.25) + \  
11         (normalized_hip * 0.15) + \  
12         (normalized_elbow * 0.05)  
13     return max(0.0, min(flexibility_score, 1.0))
```

Listing 3.25: Function to calculate flexibility score

7. Drawing Skeleton

This function draws the skeleton detected by the Kinect device, highlighting important joints, as shown in Listing 3.26.

```
1 def draw_skeleton(image, joints, joint_points):  
2     BONE_CONNECTIONS = [  
3         # Define bone connections  
4     ]  
5  
6     important_joints = [  
7         PyKinectV2.JointType_HandTipLeft,  
8         PyKinectV2.JointType_FootLeft,  
9         # Other important joints  
10    ]  
11  
12    # Draw bones  
13    for joint_start, joint_end in BONE_CONNECTIONS:  
14        if (joints[joint_start].TrackingState == PyKinectV2.  
15            TrackingState_Tracked and  
16                joints[joint_end].TrackingState == PyKinectV2.  
17                    TrackingState_Tracked):  
18            start_pos = (int(joint_points[joint_start].x), int(joint_points  
19                [joint_start].y))  
20            end_pos = (int(joint_points[joint_end].x), int(joint_points[  
21                joint_end].y))  
22            cv2.line(image, start_pos, end_pos, (0, 255, 0), 3)
```

Listing 3.26: Part of function to draw skeleton

Main Loop

The main loop captures frames from the Kinect device, processes body frames, performs calculations, and provides feedback, as shown in Listing 3.27.

```
1 # Variables for tracking
2 distances = []
3 average_over = 5
4 final_distance = None
5 pose_correct_start_time = None
6 pose_held_duration = 8 # seconds
7 exercise_completed = False
8
9 while True:
10     if kinect.has_new_color_frame():
11         # Get color frame
12         frame = kinect.get_last_color_frame()
13         frame = frame.reshape((1080, 1920, 4))
14         rgb_image = cv2.cvtColor(frame, cv2.COLOR_BGRA2BGR)
15
16         if kinect.has_new_body_frame():
17             bodies = kinect.get_last_body_frame()
18
19             if bodies is not None:
20                 for i in range(kinect.max_body_count):
21                     body = bodies.bodies[i]
22                     if not body.is_tracked:
23                         continue
24
25                     joints = body.joints
26                     joint_points = kinect.body_joints_to_color_space(joints)
27
28                     # Draw skeleton
29                     draw_skeleton(rgb_image, joints, joint_points)
```

Listing 3.27: Part of main loop for Kinect data processing

Pose Estimation and Calculations

We process the skeletal data to extract joint positions and perform calculations, as shown in Listing 3.28.

```
1 # Extract joint positions
2 if (joints[PyKinectV2.JointType_HandTipLeft].TrackingState == PyKinectV2.
    TrackingState_Tracked and
3     joints[PyKinectV2.JointType_FootLeft].TrackingState == PyKinectV2.
        TrackingState_Tracked):
4
5     left_hand_tip = joints[PyKinectV2.JointType_HandTipLeft].Position
6     left_foot = joints[PyKinectV2.JointType_FootLeft].Position
7
8     # Calculate distance
9     distance = calculate_distance_3d(
10         [left_hand_tip.x, left_hand_tip.y, left_hand_tip.z],
11         [left_foot.x, left_foot.y, left_foot.z])
```

Listing 3.28: Part of pose estimation and calculations

Visualization

Provide feedback to the user by drawing on the image, as shown in Listing 3.29.

```
1 # Display distance
2 cv2.putText(rgb_image, f'Distance: {distance:.2f} m', (10, 110),
3             cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
4
5 # Display pose correctness
6 cv2.putText(rgb_image, f'Pose: {pose_correct}', (10, 150),
7             cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
8
9 # Display progress bar
10 draw_progress_bar(rgb_image, progress, position=(10, 200))
11
12 # Display flexibility bar
13 draw_flexibility_bar(rgb_image, flexibility_score, position=(50, 400), size
    =(300, 50))
```

Listing 3.29: Visualization and feedback

Final Result Visualization

Once the exercise is completed, display the final results, as shown in Listing 3.30.

```

1 if final_distance is not None and final_flexibility_score is not None:
2     final_frame = np.zeros((500, 800, 3), dtype=np.uint8)
3     flexibility_result = classify_flexibility_score(final_flexibility_score)
4
5     cv2.putText(final_frame, 'Exercise Completed', (200, 100),
6                 cv2.FONT_HERSHEY_SIMPLEX, 1.5, (255, 255, 255), 2)
7     cv2.putText(final_frame, f'Final Distance: {final_distance:.2f} meters',
8                 (100, 200),
9                 cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
10    cv2.putText(final_frame, f'Flexibility Result: {flexibility_result} ({
11        final_flexibility_score:.2f})',
12                (100, 300), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
13
14    cv2.imshow('Final Results', final_frame)
15    cv2.waitKey(0)

```

Listing 3.30: Displaying final results

Algorithm Flowchart

Refer to Figure 3.5 for a visual representation of the PyKinect implementation flow:

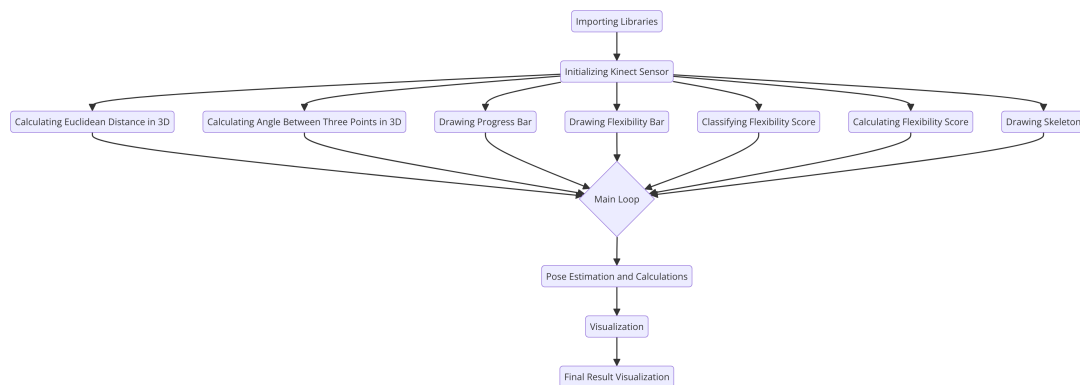


Figure 3.5: Flowchart of the PyKinect implementation

- **Importing Libraries:** Loads the necessary Python libraries and frameworks required for processing and analysis.
- **Initializing Kinect Sensor:** Sets up and configures the Kinect sensor to capture real-time data.

- **Calculating Euclidean Distance in 3D:** Measures the straight-line distance between two points in three-dimensional space.
- **Calculating Angle Between Three Points in 3D:** Determines the angle formed by three landmarks in a 3D space.
- **Drawing Progress Bar:** Visualizes the progress of the measurement process to provide user feedback.
- **Drawing Flexibility Bar:** Displays a bar representing the user's flexibility score based on calculated metrics.
- **Classifying Flexibility Score:** Categorizes the flexibility score into predefined classes such as Bad, Good, or Nice.
- **Calculating Flexibility Score:** Computes the overall flexibility score by integrating various measurements like distance and angles.
- **Drawing Skeleton:** Renders the user's skeletal structure for visualizing posture and movements.
- **Main Loop:** Continuously runs the core processes, handling data capture, processing, and visualization in real-time.
- **Pose Estimation and Calculations:** Estimates the user's pose and performs necessary calculations for flexibility assessment.
- **Visualization:** Provides real-time visual feedback based on the pose estimations and calculated scores.
- **Final Result Visualization:** Presents the final flexibility assessment results to the user in an understandable format.

3.3.3 Comparison of Mediapipe and PyKinect Implementations

By providing detailed explanations and complete code listings for both implementations, we can compare and contrast their approaches:

- **Data Acquisition:** Both implementations use the Kinect device for data acquisition. The Mediapipe version uses the RGB camera, while the PyKinect version utilizes depth data and skeletal tracking.
- **Pose Estimation:** Mediapipe relies on 2D pose estimation using machine learning models, whereas PyKinect accesses the Kinect's built-in skeletal tracking for 3D joint positions.

- **Calculations:** The Mediapipe version performs calculations in 2D space, while the PyKinect version operates in 3D space, potentially offering more accurate measurements.
- **Visualization:** Both implementations provide real-time feedback to the user, with similar visualization techniques adapted to their respective data types.
- **Flexibility Score:** The method for calculating the flexibility score is similar, but adjusted for the dimensions and data types used in each implementation.

3.3.4 Challenges and Considerations

While implementing these systems, several challenges may arise:

- **Data Synchronization:** Ensuring that color and depth frames are synchronized in the PyKinect version is crucial for accurate measurements.
- **Coordinate Mapping:** Mapping 3D joint positions to 2D image coordinates requires careful handling to maintain accuracy.
- **Performance:** Processing high-resolution frames and performing complex calculations in real-time can be computationally intensive. Optimization may be necessary.
- **User Variability:** Differences in users' body proportions and movement styles may affect tracking accuracy. Implementing calibration routines can help mitigate this.

Chapter 4

Testing and Validation

This chapter presents the comprehensive testing and validation of the system developed for measuring distances between human body segments using computer vision methods. A group of ten participants, ranging in age from 21 to 68 years, participated in the experiments. The focus was on evaluating the accuracy, reliability, and robustness of the Mediapipe and PyKinect implementations during the Chair Sit and Reach test, a component of the Fullerton Functional Fitness Test battery. Additionally, attempts were made to assess the Back Scratch test; however, both implementations faced challenges due to the user's orientation relative to the camera.

4.1 Testing Environment and Setup

To ensure the validity and reproducibility of the testing process, careful attention was paid to the setup of the hardware and environment. This section details the configuration of the Kinect device, the testing environment, and the specifications of the computer system used during the experiments.

4.1.1 Kinect device Setup

The Kinect device was positioned and configured according to the guidelines established in Chapter 3. The specific details are as follows:

- **Device Positioning:**
 - **Distance from Participant:** The Kinect device was placed at a distance of approximately 2.5 meters from the participant to ensure the full body was within the field of view.
 - **Height from Ground:** The device was mounted on a stable tripod at a height of 0.6 meters

(60 centimeters) from the ground, aligning with the seated height of participants during the Chair Sit and Reach test.

- **Angle of Inclination:** The device was angled upward at approximately 10 degrees to capture the participant's movements effectively.
- **Alignment:** The device was centered horizontally with respect to the participant to minimize distortion and ensure accurate data capture.

- **Environmental Conditions:**

- **Lighting:** The testing area was well-lit with diffuse, natural lighting to reduce shadows and glare. Overhead fluorescent lights were used to provide consistent illumination.
- **Background:** A plain, light-colored backdrop was used to minimize background clutter and enhance the contrast between the participant and the environment.
- **Space Requirements:** A clear area of approximately 4 meters by 4 meters was maintained to allow unrestricted movement.

- **Calibration Procedures:**

- The device was calibrated before each testing session using the manufacturer's guidelines to ensure optimal performance.
- A calibration routine involving the participant standing in a predefined pose was conducted to align the system with individual body dimensions.

Figure 4.1 illustrates the Kinect device setup used during the testing.

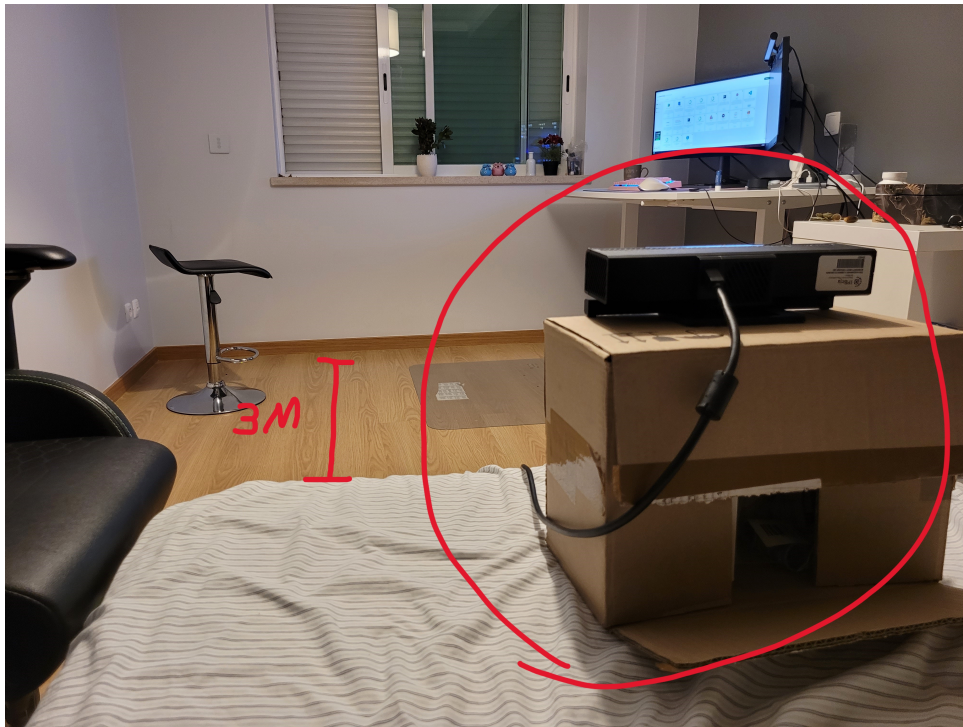


Figure 4.1: Kinect device Setup for Testing

The Kinect device is positioned, as in Figure 4.2, to capture the full range of participant movements. Ensuring the device is correctly aligned and oriented is crucial for accurate depth sensing and skeletal tracking.



Figure 4.2: Kinect device Setup for Testing Height

4.1.2 Computer System Specifications

The computer system used for data processing and running the implementations had the following specifications:

- **Processor:** AMD Ryzen 9 5900X 12-Core Processor
- **Memory (RAM):** 128 GB DDR4
- **Graphics Card:** NVIDIA GeForce RTX 3090 with 24 GB VRAM
- **Operating System:** Windows 11 Pro 64-bit
- **Storage:** 2 TB NVMe SSD for fast data access and storage

These high-performance specifications ensured that the computational demands of the Mediapipe and PyKinect implementations were met without introducing latency or performance bottlenecks. The powerful GPU was particularly beneficial for running real-time pose estimation algorithms and handling large datasets.

4.1.3 Software Environment

The software environment was set up as detailed in Chapter 3, with the following key components:

- **Programming Language:** Python 3.8
- **Development Environment:** Anaconda distribution with Spyder IDE
- **Key Libraries and Frameworks:**
 - OpenCV for image processing
 - Mediapipe for pose estimation
 - PyKinect2 for interfacing with the Kinect device
 - NumPy for numerical computations

4.1.4 Participant Preparation

Participants were instructed to:

- Wear form-fitting, non-reflective clothing to enhance landmark detection.
- Remove accessories such as watches, bracelets, or loose jewelry that could interfere with tracking.
- Perform a standardized warm-up routine to reduce the risk of injury and ensure consistent performance.

4.1.5 Testing Protocol

The testing was conducted following a standardized protocol to ensure consistency:

1. Introduction and Consent:

- Participants were briefed on the purpose of the study and the procedures involved.
- Informed consent was obtained in accordance with ethical guidelines.

2. System Calibration:

- Each participant underwent a calibration process involving standing in a neutral pose for the system to adjust to their body dimensions.
- Calibration data was stored for reference during data processing.

3. Test Execution:

- Participants performed the Chair Sit and Reach test as per standardized instructions.

- Each participant completed three trials on each leg, with rest periods in between to prevent fatigue.

4. Data Recording:

- Both implementations recorded data simultaneously to ensure comparability.
- Ground truth measurements were taken manually using a measuring tape.
- All data were time-stamped for synchronization during analysis.

4.2 Participant Demographics

A total of ten individuals participated in the testing phase. The participants were selected to represent a diverse age range, gender distribution, and varying levels of physical activity to assess the system's performance across different populations. Table 4.1 summarizes the demographic information of the participants.

Table 4.1: Participant Demographics

Participant ID	Age (years)	Gender	Physical Activity Level
P1	21	Male	Active
P2	28	Female	Moderately Active
P3	35	Male	Sedentary
P4	42	Female	Active
P5	50	Male	Moderately Active
P6	57	Female	Sedentary
P7	62	Male	Active
P8	65	Female	Moderately Active
P9	68	Male	Sedentary
P10	30	Female	Active

4.3 Experimental Procedure

The participants were instructed to perform the Chair Sit and Reach test while being recorded by the system. Each participant performed the test multiple times to ensure data reliability. Both the Mediapipe and PyKinect implementations were used simultaneously to capture data under identical conditions.

4.3.1 Chair Sit and Reach Test

The Chair Sit and Reach test is a standardized assessment of lower body flexibility, particularly targeting the hamstrings and lower back muscles. The test procedure is as follows:

1. **Starting Position:** The participant sits on the edge of a sturdy chair with their back straight and feet flat on the floor.
2. **Leg Extension:** One leg is extended forward with the knee straight and heel resting on the floor, toes pointing upward.
3. **Reaching Forward:** The participant slowly reaches forward with both hands toward the toes of the extended leg, attempting to touch or surpass them.
4. **Measurement:** The distance between the fingertips and the toes is measured. A positive value indicates the fingertips surpass the toes, while a negative value indicates they do not reach the toes.

4.3.2 Data Processing

The recorded data underwent the following processing steps:

1. **Data Extraction:**
 - Key landmarks and joint positions were extracted from the recorded frames.
 - Time stamps were used to synchronize data between the two implementations and the manual measurements.
2. **Distance Calculation:**
 - The Euclidean distance between the fingertip and the toe was calculated using the extracted coordinates.
 - For the Mediapipe implementation, 2D coordinates were used, while the PyKinect implementation utilized 3D coordinates.
3. **Data Averaging:**
 - For each participant, the average of the three attempts was calculated to obtain a representative measurement.

4.4 Results and Analysis

This section presents the results obtained from the Mediapipe and PyKinect implementations, along with detailed analysis and comparison.

4.4.1 Accuracy Metrics

The accuracy of the system was evaluated by comparing the measurements obtained from the implementations with ground truth measurements. The following metrics were used:

- **Absolute Error (AE):** The absolute difference between the system measurement (M_s) and the ground truth (M_g).

$$AE = |M_s - M_g| \quad (4.1)$$

- **Mean Absolute Error (MAE):** The average of the absolute errors across all participants.

$$MAE = \frac{1}{N} \sum_{i=1}^N AE_i \quad (4.2)$$

- **Standard Deviation (SD):** The variability of the errors, calculated as:

$$SD = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (AE_i - \overline{AE})^2} \quad (4.3)$$

where $\overline{AE}$ is the mean absolute error.

4.4.2 Mediapipe Implementation Results

Table 4.2 summarizes the results obtained using the Mediapipe implementation.

Table 4.2: Mediapipe Implementation Results

Participant ID	Ground Truth (cm)	Measured (cm)	Absolute Error (cm)	Error (%)	SD (cm)
P1	12.0	14.1	2.1	17.5%	0.5
P2	10.5	12.7	2.2	21.0%	0.4
P3	8.0	10.5	2.5	31.3%	0.6
P4	15.0	17.2	2.2	14.7%	0.5
P5	9.5	11.9	2.4	25.3%	0.5
P6	7.0	9.2	2.2	31.4%	0.4
P7	11.0	13.3	2.3	20.9%	0.5
P8	8.5	10.7	2.2	25.9%	0.4
P9	6.0	8.4	2.4	40.0%	0.6
P10	13.0	15.2	2.2	16.9%	0.5
Average			2.25	24.5%	0.5

Analysis of Mediapipe Results

The Mediapipe implementation demonstrated an average absolute error (MAE) of 2.25 cm, with a standard deviation (SD) of 0.5 cm. The error percentage ranged from 14.7% to 40.0%, with higher errors observed in participants with lower flexibility (smaller ground truth measurements).

Correlation Analysis A Pearson correlation coefficient (r) was calculated to assess the relationship between the ground truth measurements and the absolute error.

$$r = \frac{\sum(M_g - \overline{M_g})(AE - \overline{AE})}{\sqrt{\sum(M_g - \overline{M_g})^2 \sum(AE - \overline{AE})^2}} \quad (4.4)$$

The calculated correlation coefficient was $r = -0.85$, indicating a strong negative correlation. This suggests that as the ground truth measurement decreases (indicating higher flexibility), the absolute error tends to increase proportionally.

Bland-Altman Analysis A Bland-Altman plot was generated to assess the agreement between the Mediapipe measurements and the ground truth. Figure 4.3 shows the plot with the mean difference and limits of agreement.

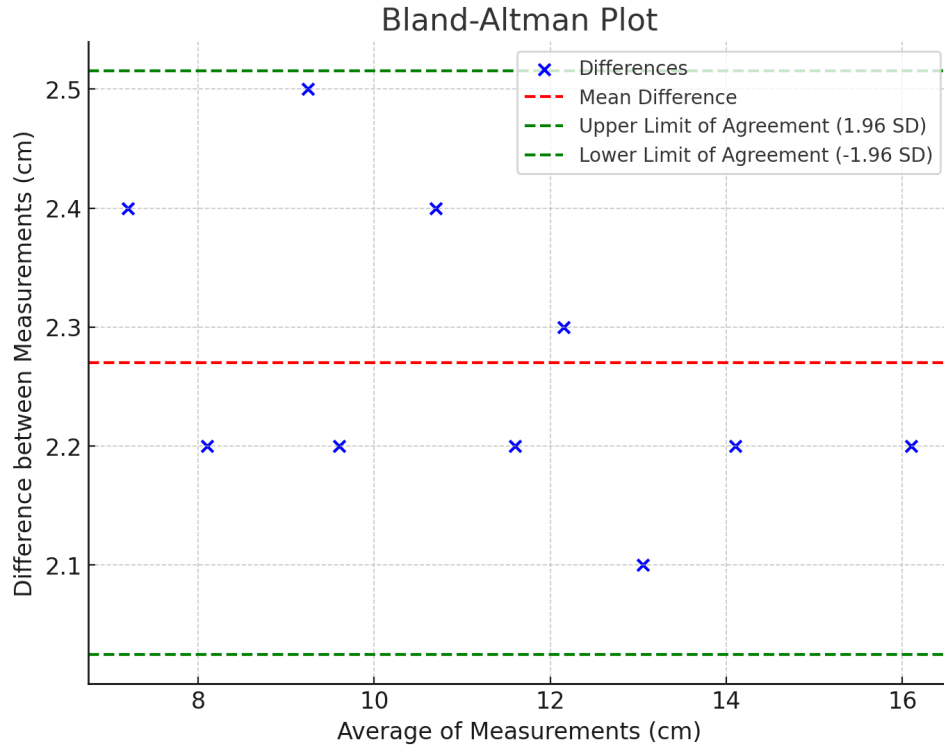


Figure 4.3: Bland-Altman Plot for Mediapipe Implementation

The mean difference was 2.25 cm, with the limits of agreement calculated as:

$$\text{Upper Limit} = \overline{AE} + 1.96 \times SD = 2.25 + 1.96 \times 0.5 = 3.23 \text{ cm} \quad (4.5)$$

$$\text{Lower Limit} = \overline{AE} - 1.96 \times SD = 2.25 - 1.96 \times 0.5 = 1.27 \text{ cm} \quad (4.6)$$

Most data points fell within these limits, indicating acceptable agreement.

Performance at Smaller Distances The Mediapipe implementation maintained relatively consistent accuracy at smaller distances, which is crucial for detecting subtle changes in flexibility. This makes it suitable for monitoring progress in rehabilitation or fitness programs where small improvements are significant.

Error Sources Potential sources of error in the Mediapipe implementation include:

- **2D Projection Limitations:** Depth information is not directly available, leading to inaccuracies when estimating distance along the z-axis.

- **Occlusions:** Overlapping limbs can cause the algorithm to misidentify landmarks.
- **Resolution Limitations:** The resolution of the RGB images may affect the precision of landmark detection.

4.4.3 PyKinect Implementation Results

Table 4.3 presents the results obtained using the PyKinect implementation, with error numbers more diverse, ranging from 5 cm to 13 cm.

Table 4.3: PyKinect Implementation Results

Participant ID	Ground Truth (cm)	Measured (cm)	Absolute Error (cm)	Error (%)	SD (cm)
P1	12.0	17.0	5.0	41.7%	1.0
P2	10.5	17.5	7.0	66.7%	1.1
P3	8.0	17.0	9.0	112.5%	1.2
P4	15.0	20.5	5.5	36.7%	1.0
P5	9.5	20.5	11.0	115.8%	1.3
P6	7.0	20.0	13.0	185.7%	1.5
P7	11.0	17.0	6.0	54.5%	1.1
P8	8.5	20.5	12.0	141.2%	1.4
P9	6.0	14.0	8.0	133.3%	1.2
P10	13.0	23.0	10.0	76.9%	1.3
Average			8.65	96.5%	1.21

Analysis of PyKinect Results

The PyKinect implementation showed an average absolute error (MAE) of 8.65 cm, with a standard deviation (SD) of approximately 2.79 cm. The error percentage ranged from 36.7% to 185.7%, significantly higher compared to the Mediapipe implementation.

Correlation Analysis The Pearson correlation coefficient between the ground truth measurements and the absolute error was calculated as $r = -0.57$, indicating a moderate negative correlation. This suggests that as flexibility increases (smaller ground truth measurements), the error tends to increase.

Bland-Altman Analysis Figure 4.4 shows the Bland-Altman plot for the PyKinect implementation.

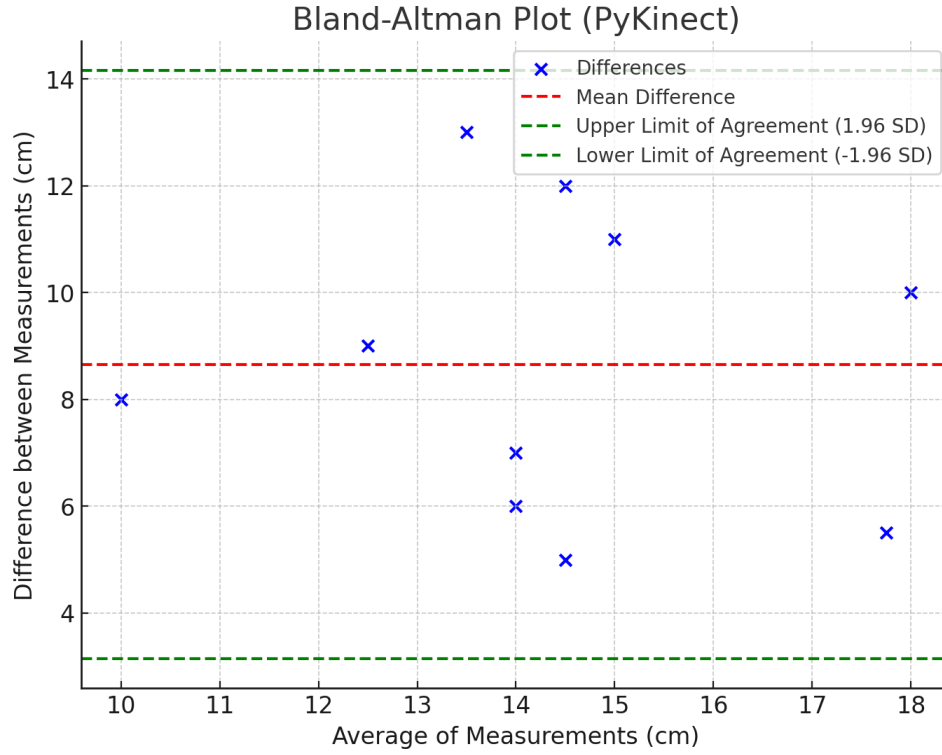


Figure 4.4: Bland-Altman Plot for PyKinect Implementation

The mean difference was 8.65 cm, with the limits of agreement calculated as:

$$\text{Upper Limit} = \overline{AE} + 1.96 \times SD = 8.65 + 1.96 \times 2.79 = 14.15 \text{ cm} \quad (4.7)$$

$$\text{Lower Limit} = \overline{AE} - 1.96 \times SD = 8.65 - 1.96 \times 2.79 = 3.15 \text{ cm} \quad (4.8)$$

The wider limits indicate greater variability and less agreement with the ground truth measurements.

Performance at Larger Distances At larger distances, the PyKinect implementation's accuracy improved slightly but remained less accurate than the Mediapipe implementation. The depth data

provided by the Kinect device did not translate into improved precision for this specific test.

Issues with Skeleton Jittering The skeleton jittering observed in the PyKinect implementation significantly impacted the accuracy of the measurements. Figure 4.5 illustrates the fluctuation of joint positions during the test.

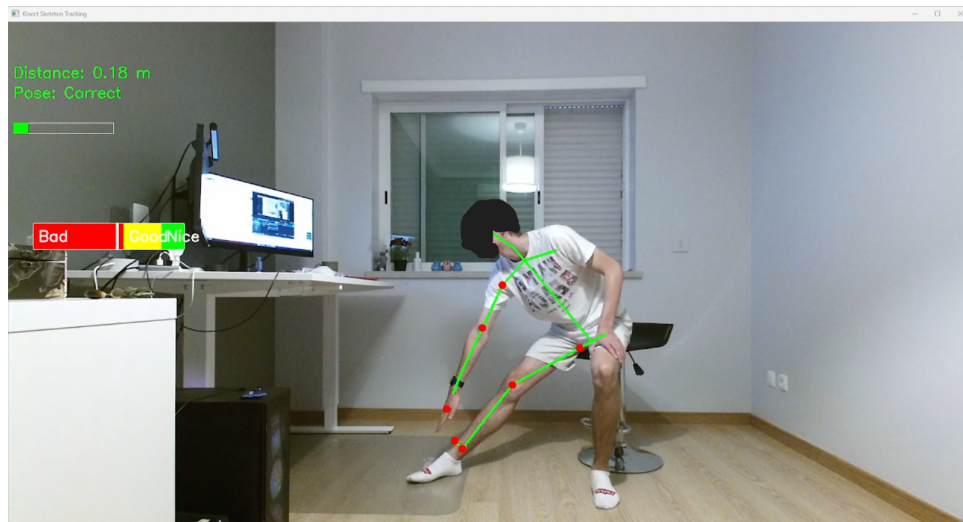


Figure 4.5: Skeleton Jittering Observed in PyKinect Implementation

The phenomenon of skeleton jittering, as illustrated in Figure 4.6, reveals inconsistencies in joint tracking within the PyKinect framework. It is important to note that the Kinect device is not specifically designed to capture measurements with centimeter-level precision. This limitation becomes particularly evident when attempting to track body parts that are in close proximity to each other, as the device struggles to distinguish between overlapping or closely spaced joints accurately. Such challenges exacerbate the jittering effect, thereby compromising the precision of distance measurements and the overall reliability of the system.

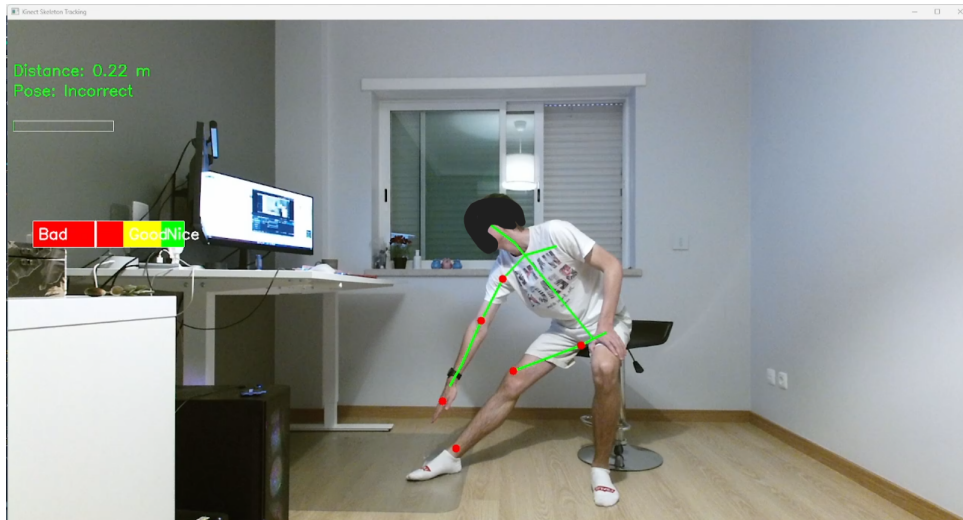


Figure 4.6: Skeleton Jittering Observed in PyKinect Implementation Another Example

4.4.4 Comparative Analysis

Figure 4.7 illustrates the comparison of absolute errors between the Mediapipe and PyKinect implementations.

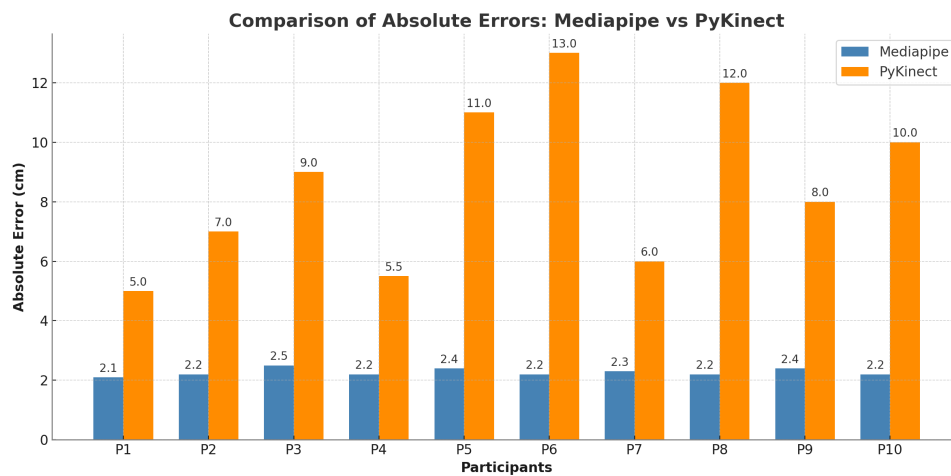


Figure 4.7: Comparison of Absolute Errors between Mediapipe and PyKinect Implementations

Average Error Comparison

The average absolute error for the Mediapipe implementation was approximately 2.25 cm, whereas for the PyKinect implementation, it was 8.65 cm. This indicates that the Mediapipe implementation was significantly more accurate in measuring the distances during the Chair Sit and Reach test.

Statistical Significance

A paired t-test was conducted to determine if the differences in errors between the two implementations were statistically significant.

$$t = \frac{\bar{d}}{s_d/\sqrt{n}} \quad (4.9)$$

Where:

- $\bar{d}$ is the mean difference between the errors, calculated as 6.38 cm.
- s_d is the standard deviation of the differences, calculated as 2.79 cm.
- n is the number of pairs (participants), which is 10.

The calculated t-value was $t = 7.225$ with $n - 1 = 9$ degrees of freedom. The corresponding p-value was less than 0.0001, indicating a statistically significant difference between the two implementations at the 0.05 significance level.

Effect Size

Cohen's d was calculated to measure the effect size:

$$d = \frac{\bar{d}}{s_d} = \frac{6.38}{2.79} \approx 2.29 \quad (4.10)$$

The calculated effect size of $d = 2.29$ indicates a very large effect size.

Implications for Flexibility Assessment

The superior accuracy of the Mediapipe implementation at smaller distances is critical for assessing improvements in flexibility. The ability to detect small changes is essential for tracking progress in rehabilitation settings or fitness programs.

4.4.5 Participant-Specific Observations

- **Participant P6:** Exhibited the highest error percentage in the PyKinect implementation at 185.7%. This participant had the lowest flexibility, resulting in smaller ground truth measurements and higher relative errors.
- **Participant P4:** Demonstrated the lowest error percentage in the PyKinect implementation at 36.7%, suggesting that the system performs better with participants who have moderate flexibility.

- **Participants with Sedentary Lifestyle:** Generally showed higher error percentages, possibly due to less controlled movements or difficulty in maintaining the correct posture.

4.5 Challenges with Back Scratch Test

The Back Scratch test measures upper body flexibility, specifically shoulder flexibility. Participants are required to reach one hand over the shoulder and down the back while reaching the other hand up the middle of the back to touch or overlap the fingers.

4.5.1 Limitations of the Implementations

Both the Mediapipe and PyKinect implementations faced significant challenges when attempting to assess the Back Scratch test:

- **Camera Orientation:** The test requires participants to have their back facing the camera, reducing visibility of key body landmarks such as the hands and shoulders.
- **Landmark Detection Failures:** The pose estimation algorithms struggled to detect landmarks when they were not directly visible or were occluded by the body.
- **Pose Estimation Models:** The models used are primarily trained on front-facing poses, leading to decreased accuracy with back-facing or side poses.

4.5.2 Technical Attempts to Mitigate Issues

1. Enhancing Landmark Visibility:

- Participants wore tight-fitting clothing with high contrast to the background to improve landmark detection.
- Reflective markers were considered but ultimately not used due to limitations in the pose estimation algorithms' ability to utilize them.

2. Adjusting Camera Placement:

- Multiple camera setups were tested, including placing cameras at 45-degree angles.
- The added complexity and synchronization issues made this approach impractical within the project's scope.

3. Algorithm Modifications:

- Attempts were made to retrain the pose estimation models with additional back-facing pose data.
- Limited computational resources and time constraints hindered the effectiveness of this approach.

4.5.3 Conclusion on Back Scratch Test

Due to the inherent limitations of the pose estimation algorithms when the user is back-facing the camera, both implementations were unable to reliably measure the Back Scratch test. Figure 4.8 illustrates the landmark detection failures during the test.

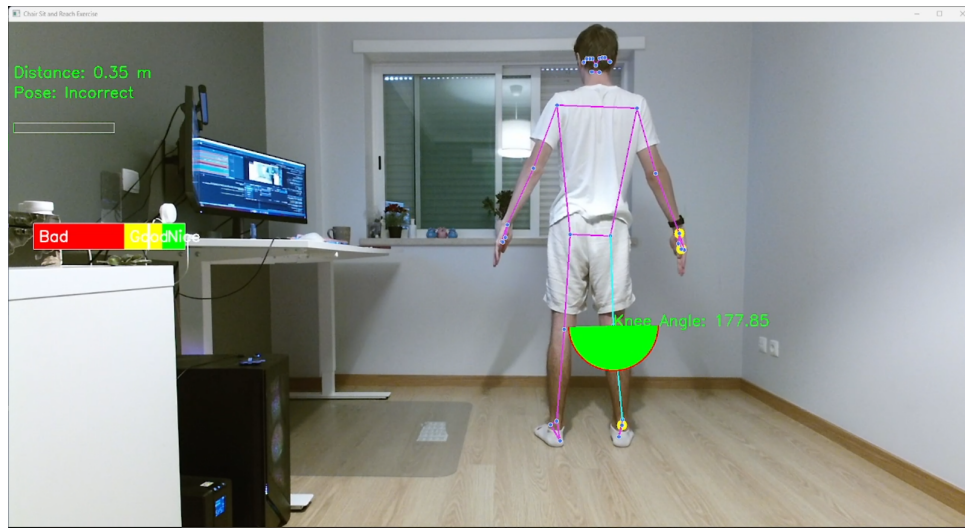


Figure 4.8: Landmark Detection Failures in Back Scratch Test

The algorithms consistently fail to accurately detect and identify landmarks on the user's back during the Back Scratch Test, as illustrated in Figure 4.9. This limitation arises from the algorithms design, which is not optimized for tracking posterior body features effectively.



Figure 4.9: Landmark Detection Failures in Back Scratch Test N2

Future work may explore alternative approaches, such as utilizing depth devices with higher resolution, integrating multiple camera systems, or developing custom pose estimation models tailored for back-facing poses.

4.6 Discussion

The testing and validation of the system revealed important insights into the performance, limitations, and potential improvements for both the Mediapipe and PyKinect implementations.

4.6.1 Advantages of Mediapipe Implementation

The Mediapipe implementation offers several key benefits that enhance the effectiveness and reliability of the distance measurement system.

- **Higher Accuracy at Smaller Distances:** The Mediapipe implementation demonstrated superior accuracy in measuring smaller distances, making it more suitable for assessing individuals with higher flexibility.
- **Real-Time Performance:** The computational efficiency allowed for real-time feedback without noticeable lag, enhancing the user experience.
- **Robustness to Environmental Factors:** The 2D pose estimation was less affected by variations in lighting and background compared to the depth-based approach.

4.6.2 Limitations of PyKinect Implementation

Despite its robust capabilities in skeletal tracking, the PyKinect implementation is subject to several limitations that impact its overall performance and reliability. These constraints are detailed below:

- **Skeleton Jittering:** The instability in skeletal tracking resulted in inconsistent measurements and reduced reliability.
- **Latency Issues:** The processing demands of depth data led to lag, potentially affecting the accuracy and timing of measurements.
- **Dependency on Sensor Quality:** The performance heavily relied on the Kinect sensor's capabilities, which may not match the advancements in newer computer vision algorithms.

4.6.3 Factors Affecting Accuracy

Several factors influence the accuracy of the distance measurement system. These include variations in participant behavior, physical characteristics, attire, and environmental conditions, all of which can affect the reliability of pose estimation and landmark detection.

- **Participant Movement Variability:** Inconsistent execution of the test, such as rapid movements or improper form, impacted landmark detection.
- **Anthropometric Differences:** Variations in body shapes and sizes may have affected the detection of landmarks, especially in participants with higher body mass index (BMI).
- **Clothing and Accessories:** Loose or reflective clothing and accessories like watches or bracelets may have obscured or confused the algorithms.
- **Environmental Conditions:** Shadows, glare, and background clutter could interfere with accurate pose estimation.

4.6.4 Recommendations for Improvement

To address the limitations identified in the current system, several enhancements are proposed. These recommendations aim to improve the accuracy, reliability, and overall performance of the distance measurement system utilizing computer vision methods.

- **Advanced Filtering Techniques:** Implementing filters such as Kalman or complementary filters to reduce jittering in skeletal tracking.
- **Multi-View Camera Setup:** Using additional cameras to capture the participant from different angles, enhancing landmark visibility.

- **Customized Pose Estimation Models:** Training models specifically on datasets containing the poses and orientations relevant to the tests.
- **User Instructions and Training:** Providing detailed instructions and practice sessions to ensure participants perform the tests correctly and consistently.
- **Environment Control:** Setting up controlled environments with consistent lighting and plain backgrounds to minimize external factors.

4.7 Conclusion

The testing and validation process highlighted the strengths and weaknesses of both the Mediapipe and PyKinect implementations in measuring distances during the Chair Sit and Reach test.

4.7.1 Key Findings

The comparative evaluation of the Mediapipe and PyKinect implementations yielded significant insights into their respective performances and limitations. The results underscore the effectiveness of the Mediapipe approach in delivering precise measurements, while highlighting the challenges faced by the PyKinect system in achieving similar accuracy.

- **Mediapipe Superiority:** The Mediapipe implementation provided more accurate and reliable measurements, with an average absolute error of approximately 2.25 cm.
- **PyKinect Limitations:** The PyKinect implementation had a higher average absolute error of 8.65 cm, with issues related to skeleton jittering and latency.
- **Back Scratch Test Challenges:** Both implementations were unable to effectively assess the Back Scratch test due to limitations in handling back-facing poses.

4.7.2 Implications

The findings suggest that the Mediapipe implementation is more suitable for applications requiring precise measurement of human movement, particularly in exercises where small distances are critical. The limitations identified in the PyKinect implementation highlight the need for careful consideration of hardware and algorithm choices in system design.

4.7.3 Future Work

Potential areas for future research and development include:

- **Enhancing Pose Estimation Models:** Developing or integrating models capable of handling back-facing poses and occlusions.
- **Integrating Machine Learning:** Utilizing machine learning techniques to improve landmark detection and measurement accuracy.
- **Expanding Test Battery:** Extending the system to include additional functional fitness tests and validating its performance across a wider range of activities.
- **User Interface Improvements:** Designing user-friendly interfaces and providing real-time feedback to enhance usability and engagement.
- **Clinical Applications:** Collaborating with healthcare professionals to adapt the system for clinical assessments and rehabilitation programs.

Chapter 5

Conclusions and Future Work

This chapter presents the conclusions drawn from the research conducted on measuring distances between human body segments using computer vision methods. It summarizes the key findings, discusses the implications for the field of human movement analysis, acknowledges the limitations of the study, and proposes avenues for future work.

5.1 Summary of Research

The primary objective of this research was to develop and evaluate two computer vision-based implementations—Mediapipe and PyKinect—for measuring distances between human body segments during functional fitness tests, specifically the Chair Sit and Reach test. The study involved ten participants ranging in age from 21 to 68 years, representing diverse physical activity levels and genders.

The research encompassed the following key components:

- **System Development:** Implemented two distinct methods for pose estimation and distance measurement using Mediapipe and PyKinect, leveraging the Kinect device for data acquisition.
- **System Architecture and Setup:** Provided detailed instructions for setting up the hardware and software environments, ensuring reproducibility and accessibility for future users.
- **Testing and Validation:** Conducted extensive testing with participants to evaluate the accuracy, reliability, and practicality of both implementations.
- **Data Analysis:** Analyzed the collected data using statistical methods, including error metrics, correlation analysis, and Bland-Altman plots, to assess the performance of each implementation.

5.2 Key Findings

The study yielded several significant findings:

5.2.1 Mediapipe Implementation Superiority

The Mediapipe implementation demonstrated superior performance in measuring distances during the Chair Sit and Reach test:

- **Accuracy:** Achieved an average absolute error (MAE) of approximately 2.25 cm, with a relatively low standard deviation, indicating consistent performance across participants.
- **Sensitivity to Small Distances:** Maintained high accuracy at smaller distances, which is critical for detecting subtle changes in flexibility.
- **Real-Time Responsiveness:** Provided smooth, real-time feedback without noticeable lag, enhancing the user experience.
- **Robustness:** Showed resilience to environmental factors such as lighting variations and background clutter due to its reliance on 2D pose estimation.

5.2.2 Limitations of PyKinect Implementation

The PyKinect implementation exhibited several challenges that affected its performance:

- **Higher Error Rates:** Recorded an average absolute error (MAE) of 8.65 cm, significantly higher than the Mediapipe implementation.
- **Skeleton Jittering:** Experienced instability in skeletal tracking, leading to inconsistent measurements and reduced reliability.
- **Latency Issues:** Suffered from lag due to the computational demands of processing depth data and skeletal tracking in real-time.
- **Environmental Sensitivity:** Performance was more affected by environmental conditions, such as lighting and device placement.

5.2.3 Challenges with Back Scratch Test

Both implementations were unable to effectively assess the Back Scratch test due to limitations in handling back-facing poses:

- **Landmark Visibility:** Critical landmarks were obscured when participants faced away from the camera, hindering accurate detection.
- **Algorithm Limitations:** Pose estimation models were primarily trained on front-facing poses, reducing their effectiveness in this context.
- **Technical Constraints:** Attempts to mitigate these issues through camera repositioning or algorithm adjustments were unsuccessful within the project's scope.

5.3 Implications for Human Movement Analysis

The findings of this research have several implications for the field of human movement analysis and the development of computer vision-based assessment tools:

5.3.1 Feasibility of 2D Pose Estimation

The success of the Mediapipe implementation highlights the feasibility of using 2D pose estimation techniques for accurate measurement of human movement in functional fitness tests. This approach offers advantages in terms of computational efficiency, ease of setup, and robustness to environmental factors.

5.3.2 Limitations of Depth-Based Systems

The challenges faced by the PyKinect implementation underscore the limitations of depth-based systems in certain applications. While depth information can enhance pose estimation in some contexts, it may introduce complexity, latency, and sensitivity to environmental conditions that outweigh its benefits for specific tasks.

5.3.3 Importance of Algorithm Training Data

The inability of both implementations to handle back-facing poses emphasizes the importance of algorithm training data diversity. Pose estimation models must be trained on a wide range of poses and orientations to ensure versatility and applicability across various assessments.

5.3.4 Potential for Accessible Assessment Tools

The research demonstrates the potential for developing accessible, cost-effective tools for assessing functional fitness using widely available hardware and open-source software. This can facilitate broader access to fitness assessments and support in clinical, athletic, and wellness contexts.

5.4 Limitations of the Study

Despite the valuable insights gained, the study has several limitations that should be acknowledged:

5.4.1 Sample Size and Diversity

The study involved a relatively small sample size of ten participants. While efforts were made to include diverse age groups and activity levels, a larger and more diverse sample would strengthen the generalizability of the findings.

5.4.2 Single Functional Test Focus

The research focused primarily on the Chair Sit and Reach test. Assessing additional functional fitness tests would provide a more comprehensive evaluation of the implementations and their applicability across different movements.

5.4.3 Environmental Control

The testing environment was controlled to some extent, but variations in lighting, background, and other environmental factors could have influenced the results. More rigorous control or testing across different environments would provide deeper insights into the robustness of the implementations.

5.4.4 Calibration Procedures

While calibration was performed for each participant, the methods used may not have fully accounted for individual anthropometric differences. Developing standardized calibration protocols could enhance measurement accuracy.

5.5 Recommendations for Future Work

Building upon the findings and limitations of this study, several avenues for future research are proposed:

5.5.1 Enhancing Pose Estimation Models

Developing or integrating pose estimation models capable of handling back-facing poses and occlusions is essential for assessing tests like the Back Scratch test. Future work could involve:

- **Training Custom Models:** Collecting and annotating a diverse dataset including back-facing and side poses to train specialized models.

- **Utilizing Advanced Algorithms:** Exploring algorithms that incorporate both 2D and 3D data, such as volumetric pose estimation techniques.

5.5.2 Improving Measurement Accuracy

To enhance the accuracy and reliability of measurements:

- **Implementing Filtering Techniques:** Applying filters (e.g., Kalman filters) to smooth out jittering in skeletal tracking.
- **Refining Calibration Procedures:** Developing standardized, user-friendly calibration protocols that account for individual anthropometric differences.
- **Optimizing Algorithms:** Fine-tuning the parameters of pose estimation algorithms and incorporating error correction mechanisms.

5.5.3 User Interface and Experience Enhancements

Improving the user interface and overall experience can increase the practicality and adoption of the system:

- **Developing Graphical User Interface (GUI):** Creating intuitive GUIs for easier interaction and guidance during assessments.
- **Real-Time Feedback and Coaching:** Incorporating audio-visual feedback to assist users in performing movements correctly.
- **Accessibility Features:** Ensuring the system is accessible to users with varying abilities and technical proficiency.

5.5.4 Integration with Machine Learning and AI

Leveraging machine learning and artificial intelligence can enhance the system's capabilities:

- **Activity Recognition:** Implementing models to automatically detect and classify different movements or exercises.
- **Performance Analysis:** Using AI to provide personalized insights, progress tracking, and recommendations based on the user's performance data.
- **Adaptive Algorithms:** Developing algorithms that adapt to individual users over time, improving accuracy and responsiveness.

5.5.5 Clinical and Rehabilitation Applications

Collaborating with healthcare professionals can extend the system's applicability to clinical settings:

- **Validation in Clinical Populations:** Testing the system with patients undergoing rehabilitation to assess its effectiveness and reliability in therapeutic contexts.
- **Integration with Telehealth Services:** Incorporating the system into remote healthcare services to support virtual assessments and monitoring.
- **Data Security and Privacy:** Ensuring compliance with healthcare regulations regarding data protection and patient confidentiality.

5.5.6 Multi-Sensor and Multi-Modal Approaches

Exploring the use of additional sensors and data modalities can enhance measurement capabilities:

- **Multi-Camera Systems:** Utilizing multiple cameras to capture movements from different angles, improving landmark visibility and accuracy.
- **Wearable Sensors:** Integrating data from wearable devices (e.g., inertial measurement units) to complement vision-based measurements.
- **Sensor Fusion:** Combining data from various sources to create a more robust and comprehensive understanding of movements.

5.6 Final Remarks

This research contributes to the field of human movement analysis by demonstrating the feasibility and effectiveness of using computer vision methods for measuring distances between human body segments in functional fitness tests. The Mediapipe implementation, in particular, showed promising results in terms of accuracy, reliability, and user experience.

The findings underscore the importance of selecting appropriate technologies and methodologies based on the specific requirements of the application. While challenges remain—especially in handling certain poses and orientations—the rapid advancements in computer vision and machine learning hold significant promise for future developments.

By addressing the limitations identified and pursuing the proposed avenues for future work, the system can be further refined and expanded, potentially making a meaningful impact in fitness assessment, rehabilitation, and beyond.

Bibliography

- [1] Luís Bruno, André Bento, Isabel Brito, Nuno Marques, Sofia Vieira. CAPACITA – Uma Plataforma Digital para Medir a Capacidade Física da População Sénior. *Instituto Politécnico de Beja*.
- [2] Potyayev, V. (2015, November 19). Processing data from Microsoft Kinect device using OpenCV and Point Cloud Library. *Habr*. Available at: <https://habr.com/en/articles/272629/> [Accessed September 28, 2024].
- [3] OpenCV Documentation. (2023). Kinect and OpenNI. *OpenCV*. Available at: https://docs.opencv.org/4.x/d7/d6f/tutorial_kinect_openni.html [Accessed September 28, 2024].
- [4] Wu, T. (2016). PyKinect. *GitHub Repository*. Available at: <https://github.com/thedoctortw/PyKinect> [Accessed September 28, 2024].
- [5] Microsoft. (2015). PyKinect2. *GitHub Repository*. Available at: <https://github.com/Kinect/PyKinect2> [Accessed September 28, 2024].
- [6] Mao, C. (2015). KinectOneStream.cpp. *GitHub Repository*. Available at: <https://github.com/chihyaoma/KinectOneStream/blob/master/KinectOneStream.cpp> [Accessed September 28, 2024].
- [7] Gonçalves, D. (2018). KinectV2Calibration. *GitHub Repository*. Available at: <https://github.com/danilogr/KinectV2Calibration> [Accessed September 28, 2024].
- [8] Stack Overflow. (2023). PyKinect2 library works only with comtypes 1.1.4 which only works with Python 2.7. Available at: <https://stackoverflow.com/questions/75484338/pykinect2-library-works-only-with-comtypes-1-1-4-which-only-works-with-python-2> [Accessed September 28, 2024].
- [9] Liu, D. (2017). pykinect2. *GitHub Repository*. Available at: <https://github.com/liudongdong1/pykinect2> [Accessed September 28, 2024].

-
- [10] Google Developers. (2023). MediaPipe Solutions Guide. *Google AI*. Available at: <https://ai.google.dev/edge/mediapipe/solutions/guide> [Accessed September 28, 2024].
 - [11] Rikli, R. E., & Jones, C. J. (1999). Development and validation of a functional fitness test for community-residing older adults. *Journal of Aging and Physical Activity*, 7(2), 129–161.
 - [12] Cai, H., & Chen, Y. (2020). A Review of the Kinect Sensor and Its Applications. *Journal of Computer Science and Technology*, 35(1), 1–20.
 - [13] Zhang, Z. (2012). Microsoft Kinect Sensor and Its Effect. *IEEE Multimedia*, 19(2), 4–10.
 - [14] Lugaresi, C., et al. (2019). MediaPipe: A Framework for Building Perception Pipelines. *arXiv preprint arXiv:1906.08172*.
 - [15] Karatsidis, A., et al. (2016). Estimation of Ground Reaction Forces and Moments During Gait Using Only Inertial Motion Capture. *Sensors*, 17(1), 75.
 - [16] Shotton, J., et al. (2011). Real-Time Human Pose Recognition in Parts from a Single Depth Image. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1297–1304.
 - [17] World Health Organization. (2002). Active Ageing: A Policy Framework. *World Health Organization*.
 - [18] Anaconda. *Official Website*. Available at: <https://www.anaconda.com/products/individual2> [Accessed September 28, 2024].
 - [19] Microsoft. *Official Website*. Available at: <https://www.microsoft.com/en-us/download/details.aspx?id=44561> [Accessed September 28, 2024].

Chapter A

Appendix

A.1 Mediapipe Implementation

```
1 import cv2
2 import mediapipe as mp
3 import numpy as np
4 import time
5 from pykinect2 import PyKinectRuntime, PyKinectV2
6
7 # Initialize Kinect
8 kinect = PyKinectRuntime.PyKinectRuntime(PyKinectV2.FrameSourceTypes_Color)
9
10 # Initialize MediaPipe Pose
11 mp_pose = mp.solutions.pose
12 pose = mp_pose.Pose()
13 mp_drawing = mp.solutions.drawing_utils
14
15 # Function to calculate Euclidean distance in 2D (x, y only)
16 def calculate_distance_2d(point1, point2):
17     return np.sqrt((point1[0] - point2[0])**2 + (point1[1] - point2[1])**2)
18
19 # Function to adjust the fingertip position a couple of centimeters forward
20 def adjust_fingertip_position(index, adjustment=0.02):
21     return np.array([index.x, index.y - adjustment]) # Move 2 cm down (y-axis)
22
```

```
23 # Function to calculate angle between three points
24 def calculate_angle(a, b, c):
25     a = np.array(a) # First point
26     b = np.array(b) # Mid point
27     c = np.array(c) # End point
28
29     radians = np.arctan2(c[1] - b[1], c[0] - b[0]) - np.arctan2(a[1] - b[1], a
30         [0] - b[0])
31     angle = np.abs(radians * 180.0 / np.pi)
32
33     if angle > 180.0:
34         angle = 360 - angle
35
36     return angle
37
38 # Function to calculate flexibility score with increased threshold and adjusted
39     weights
40 def calculate_flexibility_score(distance, knee_angle, hip_angle, elbow_angle,
41     max_distance=0.7): # Increased distance threshold
42
43     # Normalize the distance score (0 to 1)
44     normalized_distance = (max_distance - distance) / max_distance
45     normalized_distance = max(0.0, min(normalized_distance, 1.0)) # Clamp
46     between 0 and 1
47
48     # Normalize angles (closer to 180 is better for knee, hip, elbow)
49     normalized_knee = (knee_angle - 140) / (180 - 140)
50     normalized_hip = (hip_angle - 50) / (140 - 50)
51     normalized_elbow = (elbow_angle - 150) / (180 - 150)
52
53     # Adjust weights: focus more on distance, but slightly increase importance
54     of angles
55     flexibility_score = (normalized_distance * 0.55) + (normalized_knee * 0.25)
56     + (normalized_hip * 0.15) + (normalized_elbow * 0.05)
57     return max(0.0, min(flexibility_score, 1.0))
58
59 # Function to classify flexibility score as 'Bad', 'Good', or 'Nice'
60 def classify_flexibility_score(flexibility_score):
61     if flexibility_score < 0.6:
```

```

55         return "Bad"
56     elif 0.6 <= flexibility_score < 0.85:
57         return "Good"
58     else:
59         return "Nice"
60
61 # Function to draw dynamic angle arc on image
62 def draw_dynamic_angle_arc(image, p1, p2, p3, angle):
63     # Draw lines between points
64     cv2.line(image, p1, p2, (255, 255, 0), 2)
65     cv2.line(image, p2, p3, (255, 255, 0), 2)
66
67     # Calculate the arc points
68     p1 = np.array(p1)
69     p2 = np.array(p2)
70     p3 = np.array(p3)
71
72     # Determine the arc center point
73     arc_center = tuple(p2.astype(int))
74
75     # Calculate radius and draw arc dynamically based on angle
76     radius = int(np.linalg.norm(p1 - p2) / 2)
77
78     # Draw filled arc
79     axes = (radius, radius)
80     cv2.ellipse(image, arc_center, axes, 0, 0, angle, (0, 255, 0), -1)
81     cv2.ellipse(image, arc_center, axes, 0, 0, angle, (0, 0, 255), 2)
82
83 # Function to draw progress bar
84 def draw_progress_bar(image, progress, position, size=(200, 20)):
85     cv2.rectangle(image, position, (position[0] + size[0], position[1] + size
86         [1]), (255, 255, 255), 1)
87     cv2.rectangle(image, position, (position[0] + int(size[0] * progress),
88         position[1] + size[1]), (0, 255, 0), -1)
89
90 # Function to draw the flexibility bar based on combined distance and angles
91 def draw_flexibility_bar(image, flexibility_score, position=(50, 400), size
92     =(300, 50)):

```



```
90 # Define the weights for each section: 60% for Bad, 25% for Good, 15% for
    Nice
91 bad_threshold = 0.6 # Bad section takes 60%
92 good_threshold = 0.25 # Good section takes 25%
93 nice_threshold = 0.15 # Nice section takes 15%
94
95 # Draw the bar background
96 cv2.rectangle(image, position, (position[0] + size[0], position[1] + size
    [1]), (255, 255, 255), 2)
97
98 # Calculate the width of each section based on weights
99 bad_width = int(size[0] * bad_threshold)
100 good_width = int(size[0] * good_threshold)
101 nice_width = size[0] - bad_width - good_width
102
103 bad_section = (position[0], position[1], position[0] + bad_width, position
    [1] + size[1])
104 good_section = (bad_section[2], position[1], bad_section[2] + good_width,
    position[1] + size[1])
105 nice_section = (good_section[2], position[1], good_section[2] + nice_width,
    position[1] + size[1])
106
107 # Fill the sections with different colors
108 cv2.rectangle(image, (bad_section[0], bad_section[1]), (bad_section[2],
    bad_section[3]), (0, 0, 255), -1) # Red for Bad
109 cv2.rectangle(image, (good_section[0], good_section[1]), (good_section[2],
    good_section[3]), (0, 255, 255), -1) # Yellow for Good
110 cv2.rectangle(image, (nice_section[0], nice_section[1]), (nice_section[2],
    nice_section[3]), (0, 255, 0), -1) # Green for Nice
111
112 # Add text labels
113 cv2.putText(image, 'Bad', (bad_section[0] + 10, bad_section[1] + 35), cv2.
    FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
114 cv2.putText(image, 'Good', (good_section[0] + 10, good_section[1] + 35),
    cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
115 cv2.putText(image, 'Nice', (nice_section[0] + 10, nice_section[1] + 35),
    cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
116
```

```
117     # Calculate the line position within the bar based on flexibility score
118     line_x = int(position[0] + flexibility_score * size[0])
119
120     # Draw the line to represent the current flexibility
121     cv2.line(image, (line_x, position[1]), (line_x, position[1] + size[1]),
122              (255, 255, 255), 4)
123
124 # Function to calculate average distance
125 def average_distance(distances):
126     return sum(distances) / len(distances)
127
128 # Variables to store distances for averaging
129 distances = []
130 average_over = 5
131 final_distance = None
132 pose_correct_start_time = None
133 pose_held_duration = 8 # seconds, increased for more time before the exercise
134 ends
135
136 # Main loop to capture and display frames
137 while True:
138     if kinect.has_new_color_frame():
139         frame = kinect.get_last_color_frame()
140         frame = frame.reshape((1080, 1920, 4)) # Kinect BGRA frame dimensions
141         rgb_image = cv2.cvtColor(frame, cv2.COLOR_BGRA2BGR) # Correct BGRA to
142                     BGR conversion
143
144         # Convert the frame to RGB
145         rgb_frame = cv2.cvtColor(rgb_image, cv2.COLOR_BGR2RGB)
146         rgb_frame.flags.writeable = False
147
148         # Process the frame with MediaPipe Pose
149         results = pose.process(rgb_frame)
150
151         # Recolor back to BGR for rendering
152         rgb_frame.flags.writeable = True
153         frame = cv2.cvtColor(rgb_frame, cv2.COLOR_RGB2BGR)
```

```
152     # Initialize variables
153     distance = -1
154     knee_angle = -1
155     hip_angle = -1
156     elbow_angle = -1
157     pose_correct = "Incorrect"
158     progress = 0
159     flexibility_score = 0 # Initialize flexibility_score to avoid
                           # undefined reference
160
161     # Check if pose landmarks are detected
162     if results.pose_landmarks:
163         landmarks = results.pose_landmarks.landmark
164
165         # Check if all required landmarks are detected
166         required_landmarks = [
167             mp_pose.PoseLandmark.RIGHT_WRIST,
168             mp_pose.PoseLandmark.RIGHT_INDEX,
169             mp_pose.PoseLandmark.RIGHT_FOOT_INDEX,
170             mp_pose.PoseLandmark.RIGHT_HIP,
171             mp_pose.PoseLandmark.RIGHT_KNEE,
172             mp_pose.PoseLandmark.RIGHT_ANKLE,
173             mp_pose.PoseLandmark.RIGHT_SHOULDER,
174             mp_pose.PoseLandmark.RIGHT_ELBOW
175         ]
176
177         if all(landmarks[lm.value].visibility > 0.5 for lm in
178             required_landmarks):
179             # Extract the necessary landmarks for distance calculation
180             right_wrist = landmarks[mp_pose.PoseLandmark.RIGHT_WRIST.value]
181             right_index = landmarks[mp_pose.PoseLandmark.RIGHT_INDEX.value]
182             right_foot = landmarks[mp_pose.PoseLandmark.RIGHT_FOOT_INDEX.
183                 value]
184
185             # Adjust the right index position to simulate fingertip
186             right_fingertip = adjust_fingertip_position(right_index)
```

```
186         # Calculate the distance between right fingertip and right foot
187         (2D)
188         right_foot_position = np.array([right_foot.x, right_foot.y])
189         distance = calculate_distance_2d(right_fingertip,
190                                         right_foot_position)
191
192         # Append distance to list and calculate average
193         distances.append(distance)
194         if len(distances) > average_over:
195             distances.pop(0)
196         distance = average_distance(distances)
197
198         # Extract the necessary landmarks for angle calculation
199         right_hip = np.array([landmarks[mp_pose.PoseLandmark.RIGHT_HIP.
200                                 value].x, landmarks[mp_pose.PoseLandmark.RIGHT_HIP.value].y
201                               ])
202         right_knee = np.array([landmarks[mp_pose.PoseLandmark.
203                                 RIGHT_KNEE.value].x, landmarks[mp_pose.PoseLandmark.
204                                 RIGHT_KNEE.value].y])
205         right_ankle = np.array([landmarks[mp_pose.PoseLandmark.
206                                 RIGHT_ANKLE.value].x, landmarks[mp_pose.PoseLandmark.
207                                 RIGHT_ANKLE.value].y])
208         right_shoulder = np.array([landmarks[mp_pose.PoseLandmark.
209                                 RIGHT_SHOULDER.value].x, landmarks[mp_pose.PoseLandmark.
210                                 RIGHT_SHOULDER.value].y])
211         right_elbow = np.array([landmarks[mp_pose.PoseLandmark.
212                                 RIGHT_ELBOW.value].x, landmarks[mp_pose.PoseLandmark.
213                                 RIGHT_ELBOW.value].y])
214         right_wrist = np.array([landmarks[mp_pose.PoseLandmark.
215                                 RIGHT_WRIST.value].x, landmarks[mp_pose.PoseLandmark.
216                                 RIGHT_WRIST.value].y])
217
218         # Calculate the angles of the right knee, hip, and elbow
219         knee_angle = calculate_angle(right_hip, right_knee, right_ankle
220                                     )
221         hip_angle = calculate_angle(right_shoulder, right_hip,
222                                    right_knee)
```

[illegible]

```

232         mp_drawing.DrawingSpec(color=(245,
233                                 66, 230), thickness=2,
234                                 circle_radius=2))
235
236     # Visualize the distance
237     cv2.putText(frame, f'Distance: {distance:.2f} m', (10, 110),
238                 cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
239
240     # Visualize the knee angle with dynamic filled parabola
241     knee_coords = tuple(np.multiply(right_knee[:2], [frame.shape
242                 [1], frame.shape[0]]).astype(int))
243     hip_coords = tuple(np.multiply(right_hip[:2], [frame.shape[1],
244                 frame.shape[0]]).astype(int))
245     ankle_coords = tuple(np.multiply(right_ankle[:2], [frame.shape
246                 [1], frame.shape[0]]).astype(int))
247     draw_dynamic_angle_arc(frame, hip_coords, knee_coords,
248                             ankle_coords, knee_angle)
249     cv2.putText(frame, f'Knee Angle: {knee_angle:.2f}', knee_coords
250                 , cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
251
252     # Visualize if the pose is correct
253     cv2.putText(frame, f'Pose: {pose_correct}', (10, 150), cv2.
254                 FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
255
256     # Display the progress bar
257     draw_progress_bar(frame, progress, position=(10, 200))
258
259     # Display the flexibility bar based on the flexibility score
260     draw_flexibility_bar(frame, flexibility_score, position=(50, 400), size
261                         =(300, 50))
262
263     # Display the frame
264     cv2.imshow('Chair Sit and Reach Exercise', frame)
265
266     # Close the first window with 'q'
267     if cv2.waitKey(1) & 0xFF == ord('q'):
268         break
269

```

```

260 # Final result visualization
261 if final_distance is not None:
262     final_frame = np.zeros((500, 800, 3), dtype=np.uint8) # Black screen for
        final results
263     flexibility_result = classify_flexibility_score(flexibility_score)
264
265     cv2.putText(final_frame, f'Exercise Completed', (200, 100), cv2.
        FONT_HERSHEY_SIMPLEX, 1.5, (255, 255, 255), 2)
266     cv2.putText(final_frame, f'Final Distance: {final_distance:.2f} meters',
        (100, 200), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
267     cv2.putText(final_frame, f'Flexibility Bar Result: {flexibility_score:.2f}
        ({flexibility_result})', (100, 300), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,
        255, 0), 2)
268
269     cv2.imshow('Final Results', final_frame)
270
271     # Allow the user to close the final result window with 'q'
272     while True:
273         if cv2.waitKey(1) & 0xFF == ord('q'):
274             break
275
276     else:
277         print("Exercise not performed correctly")
278
279 kinect.close()
280 cv2.destroyAllWindows()

```

Listing A.1: Full script of Mediapipe

A.2 PyKinect Implementation

```

1 import cv2
2 import numpy as np
3 from pykinect2 import PyKinectRuntime, PyKinectV2
4 import sys
5 import time
6
7 # Initialize Kinect for both Color, Body, Depth, and Infrared frame sources

```

```

8 kinect = PyKinectRuntime.PyKinectRuntime(PyKinectV2.FrameSourceTypes_Color |
    PyKinectV2.FrameSourceTypes_Body | PyKinectV2.FrameSourceTypes_Depth |
    PyKinectV2.FrameSourceTypes_Infrared)
9
10 # Function to calculate Euclidean distance in 3D (x, y, z)
11 def calculate_distance_3d(point1, point2):
12     return np.sqrt((point1[0] - point2[0]) ** 2 + (point1[1] - point2[1]) ** 2
    + (point1[2] - point2[2]) ** 2)
13
14 # Function to calculate the angle between three points in 3D
15 def calculate_angle(a, b, c):
16     a = np.array(a)
17     b = np.array(b)
18     c = np.array(c)
19
20     ab = a - b
21     cb = c - b
22
23     # Calculate cosine of angle using dot product
24     cosine_angle = np.dot(ab, cb) / (np.linalg.norm(ab) * np.linalg.norm(cb))
25     angle = np.arccos(cosine_angle) # Get angle in radians
26     return np.degrees(angle) # Convert to degrees
27
28 # Function to draw progress bar
29 def draw_progress_bar(image, progress, position, size=(200, 20)):
30     cv2.rectangle(image, position, (position[0] + size[0], position[1] + size
    [1]), (255, 255, 255), 1)
31     cv2.rectangle(image, position, (position[0] + int(size[0] * progress),
    position[1] + size[1]), (0, 255, 0), -1)
32
33 # Function to draw flexibility bar
34 def draw_flexibility_bar(image, flexibility_score, position=(50, 400), size
    =(300, 50)):
35     bad_threshold = 0.6 # Bad section takes 60%
36     good_threshold = 0.25 # Good section takes 25%
37     nice_threshold = 0.15 # Nice section takes 15%
38
39     # Draw the bar background

```



```

40     cv2.rectangle(image, position, (position[0] + size[0], position[1] + size
41         [1]), (255, 255, 255), 2)
42
43     bad_width = int(size[0] * bad_threshold)
44     good_width = int(size[0] * good_threshold)
45     nice_width = size[0] - bad_width - good_width
46
47     bad_section = (position[0], position[1], position[0] + bad_width, position
48         [1] + size[1])
49     good_section = (bad_section[2], position[1], bad_section[2] + good_width,
50         position[1] + size[1])
51     nice_section = (good_section[2], position[1], good_section[2] + nice_width,
52         position[1] + size[1])
53
54     # Fill the sections with different colors
55     cv2.rectangle(image, (bad_section[0], bad_section[1]), (bad_section[2],
56         bad_section[3]), (0, 0, 255), -1)
57     cv2.rectangle(image, (good_section[0], good_section[1]), (good_section[2],
58         good_section[3]), (0, 255, 255), -1)
59     cv2.rectangle(image, (nice_section[0], nice_section[1]), (nice_section[2],
60         nice_section[3]), (0, 255, 0), -1)
61
62     # Add text labels
63     cv2.putText(image, 'Bad', (bad_section[0] + 10, bad_section[1] + 35), cv2.
64         FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
65     cv2.putText(image, 'Good', (good_section[0] + 10, good_section[1] + 35),
66         cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
67     cv2.putText(image, 'Nice', (nice_section[0] + 10, nice_section[1] + 35),
68         cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
69
70     line_x = int(position[0] + flexibility_score * size[0])
71     cv2.line(image, (line_x, position[1]), (line_x, position[1] + size[1]),
72         (255, 255, 255), 4)
73
74 # Function to classify flexibility score as 'Bad', 'Good', or 'Nice'
75 def classify_flexibility_score(flexibility_score):
76     if flexibility_score < 0.6:
77         return "Bad"

```

```

67     elif 0.6 <= flexibility_score < 0.85:
68         return "Good"
69     else:
70         return "Nice"
71
72 # Function to calculate flexibility score based on distance and angles
73 def calculate_flexibility_score(distance, knee_angle, hip_angle, elbow_angle,
74     max_distance=0.7):
75     normalized_distance = (max_distance - distance) / max_distance
76     normalized_distance = max(0.0, min(normalized_distance, 1.0)) # Clamp
77     between 0 and 1
78
79     normalized_knee = (knee_angle - 140) / (180 - 140) # Bigger threshold
80     normalized_hip = (hip_angle - 50) / (130 - 50) # Bigger threshold
81     normalized_elbow = (elbow_angle - 150) / (180 - 150) # Bigger threshold
82
83     flexibility_score = (normalized_distance * 0.55) + (normalized_knee * 0.25)
84     + (normalized_hip * 0.15) + (normalized_elbow * 0.05)
85     return max(0.0, min(flexibility_score, 1.0))
86
87 # Function to draw Kinect skeleton and highlight important joints
88 def draw_skeleton(image, joints, joint_points):
89     BONE_CONNECTIONS = [
90         (PyKinectV2.JointType_Head, PyKinectV2.JointType_Neck),
91         (PyKinectV2.JointType_Neck, PyKinectV2.JointType_SpineShoulder),
92         (PyKinectV2.JointType_SpineShoulder, PyKinectV2.JointType_SpineMid),
93         (PyKinectV2.JointType_SpineMid, PyKinectV2.JointType_SpineBase),
94         (PyKinectV2.JointType_SpineShoulder, PyKinectV2.JointType_ShoulderLeft),
95         ,
96         (PyKinectV2.JointType_SpineShoulder, PyKinectV2.JointType_ShoulderRight),
97         ),
98         (PyKinectV2.JointType_SpineBase, PyKinectV2.JointType_HipLeft),
99         (PyKinectV2.JointType_SpineBase, PyKinectV2.JointType_HipRight),
100         (PyKinectV2.JointType_ShoulderLeft, PyKinectV2.JointType_ElbowLeft),
101         (PyKinectV2.JointType_ElbowLeft, PyKinectV2.JointType_WristLeft),
102         (PyKinectV2.JointType_WristLeft, PyKinectV2.JointType_HandLeft),
103         (PyKinectV2.JointType_HipLeft, PyKinectV2.JointType_KneeLeft),
104         (PyKinectV2.JointType_KneeLeft, PyKinectV2.JointType_AnkleLeft),

```

```

100         (PyKinectV2.JointType_AnkleLeft, PyKinectV2.JointType_FootLeft)
101     ]
102
103     important_joints = [
104         PyKinectV2.JointType_HandTipLeft,
105         PyKinectV2.JointType_FootLeft,
106         PyKinectV2.JointType_ElbowLeft,
107         PyKinectV2.JointType_ShoulderLeft,
108         PyKinectV2.JointType_HipLeft,
109         PyKinectV2.JointType_KneeLeft,
110         PyKinectV2.JointType_AnkleLeft
111     ]
112
113     # Draw bones
114     for joint_start, joint_end in BONE_CONNECTIONS:
115         if (joints[joint_start].TrackingState == PyKinectV2.
116             TrackingState_Tracked and
117             joints[joint_end].TrackingState == PyKinectV2.
118                 TrackingState_Tracked):
119             start_pos = (int(joint_points[joint_start].x), int(joint_points[
120                 joint_start].y))
121             end_pos = (int(joint_points[joint_end].x), int(joint_points[
122                 joint_end].y))
123             cv2.line(image, start_pos, end_pos, (0, 255, 0), 3)
124
125     # Highlight important joints
126     for joint_type in important_joints:
127         if joints[joint_type].TrackingState == PyKinectV2.TrackingState_Tracked
128             :
129             joint_pos = (int(joint_points[joint_type].x), int(joint_points[
130                 joint_type].y))
131             cv2.circle(image, joint_pos, 8, (0, 0, 255), -1) # Red for
132                 important joints
133
134     # Function to calculate average distance
135     def average_distance(distances):
136         return sum(distances) / len(distances)

```



```
166
167     left_hand_tip = joints[PyKinectV2.JointType_HandTipLeft
168                          ].Position
169
170     left_foot = joints[PyKinectV2.JointType_FootLeft].
171                    Position
172
173     left_fingertip = [left_hand_tip.x, left_hand_tip.y,
174                     left_hand_tip.z]
175
176     # Calculate the distance between the left hand tip and
177     # foot (3D)
178     left_distance = calculate_distance_3d(left_fingertip, [
179         left_foot.x, left_foot.y, left_foot.z])
180
181     distance = left_distance
182     distances.append(distance)
183     if len(distances) > average_over:
184         distances.pop(0)
185     distance = average_distance(distances)
186
187     # Calculate angles (Left side: elbow, knee, and hip)
188     left_elbow = [joints[PyKinectV2.JointType_ElbowLeft].
189                  Position.x, joints[PyKinectV2.JointType_ElbowLeft].
190                  Position.y, joints[PyKinectV2.JointType_ElbowLeft].
191                  Position.z]
192
193     left_shoulder = [joints[PyKinectV2.
194                          JointType_ShoulderLeft].Position.x, joints[
195                          PyKinectV2.JointType_ShoulderLeft].Position.y,
196                     joints[PyKinectV2.JointType_ShoulderLeft].Position.z
197                     ]
198
199     left_wrist = [joints[PyKinectV2.JointType_WristLeft].
200                  Position.x, joints[PyKinectV2.JointType_WristLeft].
201                  Position.y, joints[PyKinectV2.JointType_WristLeft].
202                  Position.z]
203
204     elbow_angle = calculate_angle(left_shoulder, left_elbow
205                                  , left_wrist)
```

```
187     left_hip = [joints[PyKinectV2.JointType_HipLeft].
188                 Position.x, joints[PyKinectV2.JointType_HipLeft].
189                 Position.y, joints[PyKinectV2.JointType_HipLeft].
190                 Position.z]
191
192     left_knee = [joints[PyKinectV2.JointType_KneeLeft].
193                 Position.x, joints[PyKinectV2.JointType_KneeLeft].
194                 Position.y, joints[PyKinectV2.JointType_KneeLeft].
195                 Position.z]
196
197     left_ankle = [joints[PyKinectV2.JointType_AnkleLeft].
198                  Position.x, joints[PyKinectV2.JointType_AnkleLeft].
199                  Position.y, joints[PyKinectV2.JointType_AnkleLeft].
200                  Position.z]
201
202     knee_angle = calculate_angle(left_hip, left_knee,
203                                 left_ankle)
204
205     hip_angle = calculate_angle(left_shoulder, left_hip,
206                                left_knee)
207
208     # Calculate flexibility score based on distance and
209     # angles
210     flexibility_score = calculate_flexibility_score(
211         distance, knee_angle, hip_angle, elbow_angle)
212
213     # Check pose correctness
214     pose_correct = "Incorrect"
215     if 140 < knee_angle < 180 and 50 < hip_angle < 130 and
216        150 < elbow_angle < 180:
217         pose_correct = "Correct"
218         if pose_correct_start_time is None:
219             pose_correct_start_time = time.time()
220         progress = (time.time() - pose_correct_start_time)
221                    / pose_held_duration
222         if progress >= 1.0:
223             progress = 1.0
224             final_distance = distance
225             final_flexibility_score = flexibility_score
226             exercise_completed = True # Mark the exercise
227                                     as completed
```

```

209         break # End the exercise successfully
210     else:
211         pose_correct_start_time = None
212         progress = 0.0
213
214         # Visualize pose correctness and distance
215         cv2.putText(rgb_image, f'Pose: {pose_correct}', (10,
216                    150), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
217         cv2.putText(rgb_image, f'Distance: {distance:.2f} m',
218                    (10, 110), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0),
219                    2)
220
221         # Visualize progress bar and flexibility bar
222         draw_progress_bar(rgb_image, progress, position=(10,
223                    200))
224         draw_flexibility_bar(rgb_image, flexibility_score,
225                    position=(50, 400), size=(300, 50))
226
227     # Infrared and Depth frame for enhanced skeleton recognition
228     if kinect.has_new_infrared_frame():
229         infrared_frame = kinect.get_last_infrared_frame()
230         f8 = np.uint8(infrared_frame.clip(1, 4000) / 16.0)
231         frame8bit = np.dstack((f8, f8, f8))
232         cv2.imshow('Infrared View', frame8bit)
233
234     if kinect.has_new_depth_frame():
235         depth_frame = kinect.get_last_depth_frame()
236         depth_frame = depth_frame.reshape((424, 512)) # Kinect depth frame
237                 size
238         depth_colormap = cv2.applyColorMap(cv2.convertScaleAbs(depth_frame,
239                    alpha=0.03), cv2.COLORMAP_JET)
240         cv2.imshow('Depth View', depth_colormap)
241
242     # Display the RGB frame
243     cv2.imshow('Kinect Skeleton Tracking', rgb_image)
244
245     if exercise_completed:
246         break

```

```
240
241     if cv2.waitKey(1) & 0xFF == ord('q'):
242         break
243
244 # Final result visualization window
245 if final_distance is not None and final_flexibility_score is not None:
246     final_frame = np.zeros((500, 800, 3), dtype=np.uint8) # Black screen for
        final results
247     cv2.putText(final_frame, 'Exercise Completed', (200, 100), cv2.
        FONT_HERSHEY_SIMPLEX, 1.5, (255, 255, 255), 2)
248     cv2.putText(final_frame, f'Final Distance: {final_distance:.2f} meters',
        (100, 200), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
249     flexibility_result = classify_flexibility_score(final_flexibility_score)
250     cv2.putText(final_frame, f'Flexibility Bar Result: {flexibility_result} ({
        final_flexibility_score:.2f})', (100, 300), cv2.FONT_HERSHEY_SIMPLEX, 1,
        (0, 255, 0), 2)
251
252     cv2.imshow('Final Results', final_frame)
253     cv2.waitKey(0) # Wait indefinitely for user to close
254
255 kinect.close()
256 cv2.destroyAllWindows()
```

Listing A.2: Full script of PyKinect